Pet Guardian

Mastering Relational and Non-Relational Database - Sprint 4

Enzo Okuizumi RM 561432

Gustavo Okada RM 563428

Lucas Barros RM 566422

Luna Guimarães RM 562290

Milton Marcelino RM 564836

Links Importantes 3
Modelo Lógico 4
Modelo Relacional 5
Tabelas Criadas no Oracle SQL Developer 6
DML - Cargas de Dados 18
Objetos PL/SQL 19
Bloco 1: Objetos PL/SQL — Empacotamento (PKG_PETGUARDIAN) e Trigger 19
Empacotamento de Processos: Package PKG_PETGUARDIAN 19
Exceções Personalizadas Tratadas no Pacote 20
Código SQL Completo (sprint4_pkg_petguardian.sql) 21
Bloco 2: Bateria de Testes e Evidências de Execução (PKG_PETGUARDIAN) 33
Resultados da Bateria de Testes Integrada (Testes 1 a 6) 33
Evidências de Execução no Oracle SQL Developer 34
Bloco 3: Banco de Dados Não Relacional — MongoDB (aulas e conteudos_aula) 36
Persistência Poliglota e Arquitetura Dual de Coleções 36
1. Coleção aulas (Metadados Relacionais Exportados) 36
1.1 Dataset Estruturado em JSON (mongodb/aulas_dataset.json) 36
1.2 Script NoSQL de Criação, Validação e Carga (mongodb/setup_mongodb_aulas.js) 37
2. Coleção conteudos_aula (Conteúdo Pedagógico Rico em Markdown) 40
2.1 Dataset Didático em JSON (mongodb/conteudos_dataset.json) 40
2.2 Script NoSQL de Criação, Validação e Carga (mongodb/setup_mongodb_conteudos.js) 42
3. Evidências de Execução e Importação no MongoDB (MONGOSH / Compass) 46

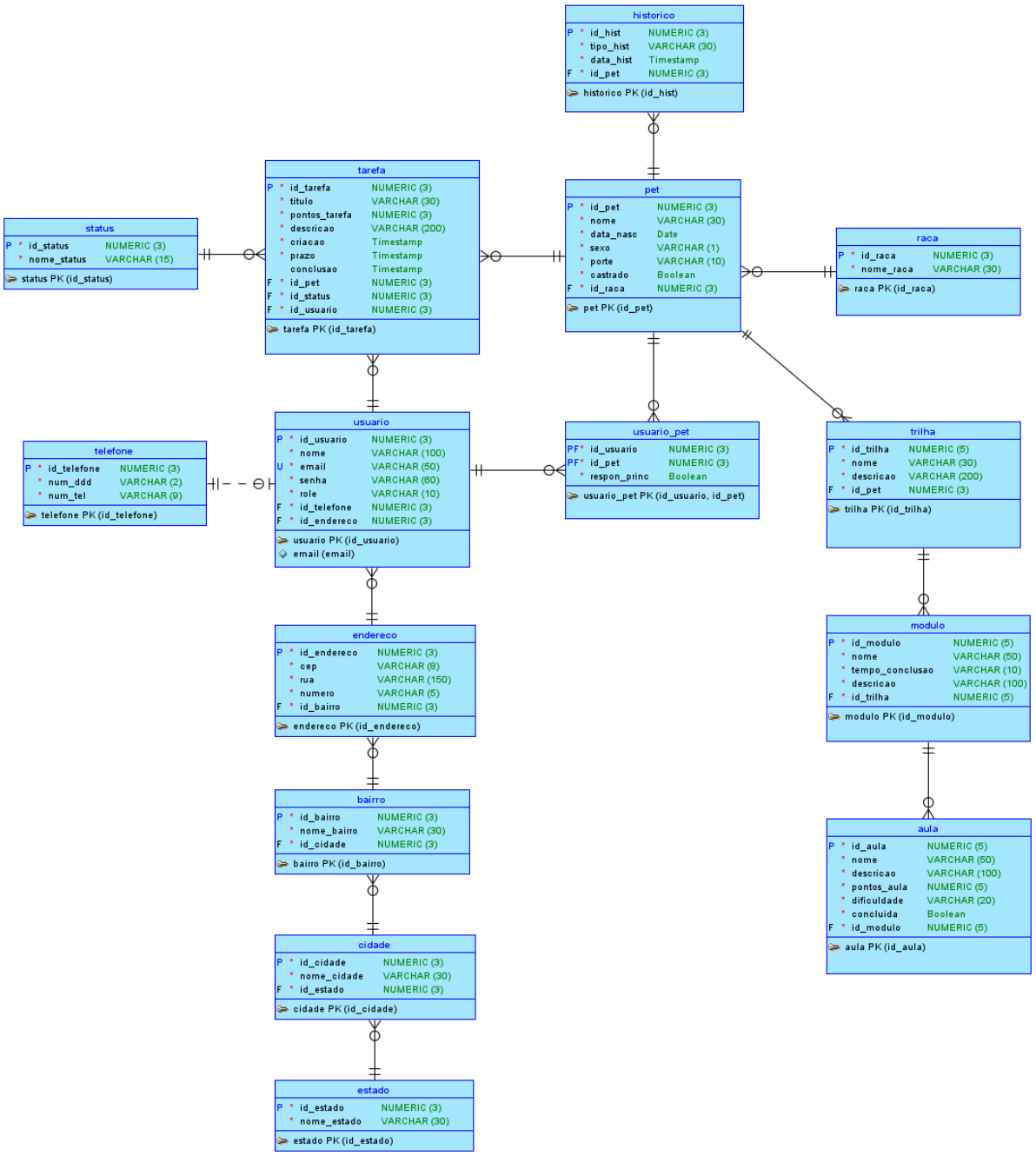
Links Importantes

Repositório Github: Challenge-Pet-Guardian-4/Database-Advanced

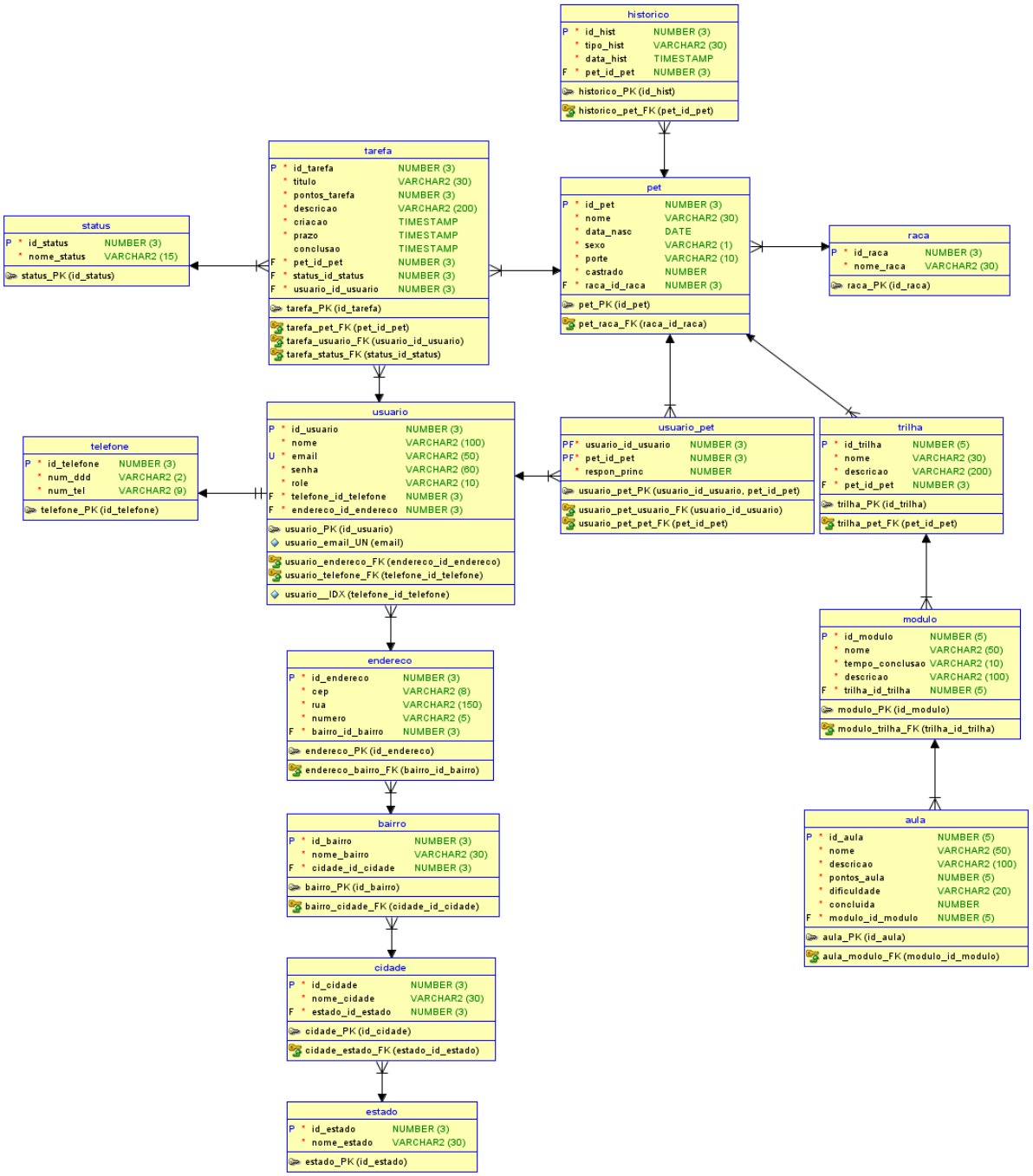
Link Vídeo Youtube:

Responsáveis pelo desenvolvimento: Lucas Barros / Enzo Okuizumi

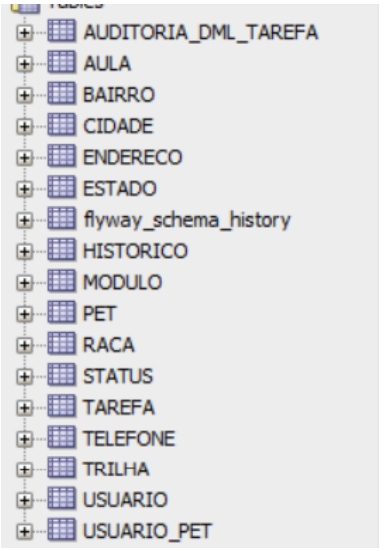
Modelo Lógico



Modelo Relacional



Tabelas Criadas no Oracle SQL Developer




```
Table TELEFONE created.


Table ESTADO created.


Table CIDADE created.


Table BAIRRO created.


Table ENDERECO created.


Table USUARIO created.


INDEX USUARIO__IDX created.


Table RACA created.

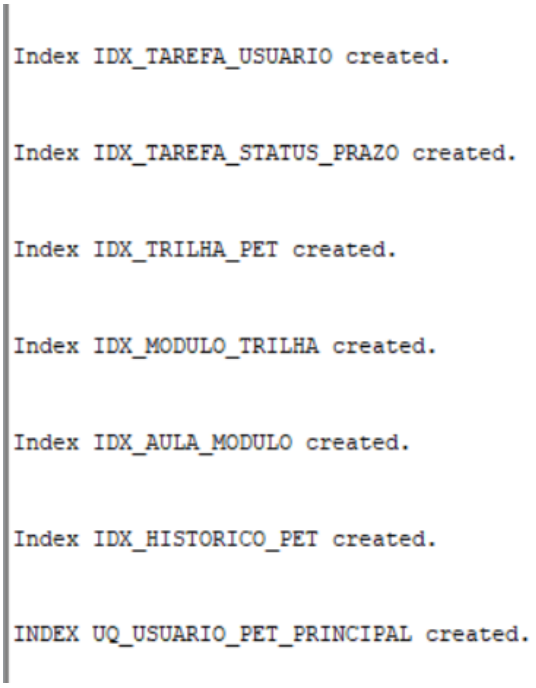

Table PET created.


Table USUARIO_PET created.


Table STATUS created.


Table TAREFA created.
```


Script Output
Task completed in 1,161 seconds
Table MODULO created.
Table AULA created.
Table HISTORICO created.
Table AUDITORIA_DML_TAREFA created.
Index IDX_CIDADE_ESTADO created.
Index IDX_BAIRRO_CIDADE created.
Index IDX_ENDERECO_BAIRRO created.
Index IDX_USUARIO_ENDERECO created.
Index IDX_PET_RACA created.
Index IDX_USUARIO_PET_PET created.
Index IDX_TAREFA_PET created.
Index IDX_TAREFA_STATUS created.


Index IDX_TAREFA_USUARIO created.
Index IDX_TAREFA_STATUS_PRAZO created.
Index IDX_TRILHA_PET created.
Index IDX_MODULO_TRILHA created.
Index IDX_AULA_MODULO created.
Index IDX_HISTORICO_PET created.
INDEX UQ_USUARIO_PET_PRINCIPAL created.

```sql
-- PARTE 1: DDL COMPLETO (challenge.ddl)
-- 15 Tabelas em 3ª Forma Normal + Tabela de Auditoria DML

-- 1. TELEFONE
CREATE TABLE telefone (
    id_telefone NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT NULL,
    num_ddd     VARCHAR2(2) NOT NULL,
    num_tel     VARCHAR2(9) NOT NULL,
    CONSTRAINT telefone_pk PRIMARY KEY (id_telefone)
);

-- 2. ESTADO
CREATE TABLE estado (
    id_estado   NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT NULL,
    nome_estado VARCHAR2(30) NOT NULL,
    CONSTRAINT estado_pk PRIMARY KEY (id_estado)
);

-- 3. CIDADE
CREATE TABLE cidade (
    id_cidade        NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT
NULL,
    nome_cidade      VARCHAR2(30) NOT NULL,
    estado_id_estado NUMBER(3) NOT NULL,
    CONSTRAINT cidade_pk PRIMARY KEY (id_cidade),
    CONSTRAINT cidade_estado_fk FOREIGN KEY (estado_id_estado) REFERENCES
estado(id_estado)
);

-- 4. BAIRRO
CREATE TABLE bairro (
    id_bairro        NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT
NULL,
    nome_bairro      VARCHAR2(30) NOT NULL,
    cidade_id_cidade NUMBER(3) NOT NULL,
    CONSTRAINT bairro_pk PRIMARY KEY (id_bairro),
    CONSTRAINT bairro_cidade_fk FOREIGN KEY (cidade_id_cidade) REFERENCES
cidade(id_cidade)
);

-- 5. ENDERECO
CREATE TABLE endereco (
    id_endereco      NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT
NULL,
    cep              VARCHAR2(8) NOT NULL,
    rua              VARCHAR2(150) NOT NULL,
    numero           VARCHAR2(5) NOT NULL,
    bairro_id_bairro NUMBER(3) NOT NULL,
```

```
    CONSTRAINT endereco_pk PRIMARY KEY (id_endereco),
    CONSTRAINT endereco_bairro_fk FOREIGN KEY (bairro_id_bairro) REFERENCES
bairro(id_bairro)
);

-- 6. USUARIO (1:N com ENDERECO via FK endereco_id_endereco)
CREATE TABLE usuario (
    id_usuario           NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT
NULL,
    nome                 VARCHAR2(100) NOT NULL,
    email                VARCHAR2(50) NOT NULL,
    senha                VARCHAR2(60) NOT NULL,
    role                 VARCHAR2(10) NOT NULL,
    telefone_id_telefone NUMBER(3) NOT NULL,
    endereco_id_endereco NUMBER(3) NOT NULL,
    CONSTRAINT usuario_pk PRIMARY KEY (id_usuario),
    CONSTRAINT usuario_email_un UNIQUE (email),
    CONSTRAINT ck_usuario_role CHECK (role IN ('ADMIN', 'COMUM', 'PREMIUM')),
    CONSTRAINT usuario_telefone_fk FOREIGN KEY (telefone_id_telefone)
REFERENCES telefone(id_telefone),
    CONSTRAINT usuario_endereco_fk FOREIGN KEY (endereco_id_endereco)
REFERENCES endereco(id_endereco)
);
CREATE UNIQUE INDEX usuario__idx ON usuario(telefone_id_telefone ASC);

-- 7. RACA
CREATE TABLE raca (
    id_raca   NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT NULL,
    nome_raca VARCHAR2(30) NOT NULL,
    CONSTRAINT raca_pk PRIMARY KEY (id_raca)
);

-- 8. PET
CREATE TABLE pet (
    id_pet       NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT NULL,
    nome         VARCHAR2(30) NOT NULL,
    data_nasc    DATE NOT NULL,
    sexo         VARCHAR2(1) NOT NULL,
    porte        VARCHAR2(10) NOT NULL,
    castrado     NUMBER NOT NULL,
    raca_id_raca NUMBER(3) NOT NULL,
    CONSTRAINT pet_pk PRIMARY KEY (id_pet),
    CONSTRAINT ck_pet_sexo CHECK (sexo IN ('F', 'M')),
    CONSTRAINT ck_pet_porte CHECK (porte IN ('GRANDE', 'MEDIO', 'PEQUENO')),
    CONSTRAINT ck_pet_castrado CHECK (castrado IN (0, 1)),
    CONSTRAINT pet_raca_fk FOREIGN KEY (raca_id_raca) REFERENCES raca(id_raca)
);
```

```sql
-- 9. USUARIO_PET (Care Circle Familiar N:M)
CREATE TABLE usuario_pet (
    usuario_id_usuario NUMBER(3) NOT NULL,
    pet_id_pet         NUMBER(3) NOT NULL,
    respon_princ       NUMBER NOT NULL,
    CONSTRAINT usuario_pet_pk PRIMARY KEY (usuario_id_usuario, pet_id_pet),
    CONSTRAINT ck_usuario_pet_princ CHECK (respon_princ IN (0, 1)),
    CONSTRAINT usuario_pet_usuario_fk FOREIGN KEY (usuario_id_usuario)
REFERENCES usuario(id_usuario),
    CONSTRAINT usuario_pet_pet_fk FOREIGN KEY (pet_id_pet) REFERENCES
pet(id_pet)
);

-- 10. STATUS (Domínio de Tarefas)
CREATE TABLE status (
    id_status   NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT NULL,
    nome_status VARCHAR2(15) NOT NULL,
    CONSTRAINT status_pk PRIMARY KEY (id_status),
    CONSTRAINT ck_status_nome CHECK (nome_status IN ('CONCLUIDO', 'EXPIRADO',
'PENDENTE'))
);

-- 11. TAREFA (Tabela de Fatos de Rotina Gamificada)
CREATE TABLE tarefa (
    id_tarefa          NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT
NULL,
    titulo             VARCHAR2(30) NOT NULL,
    pontos_tarefa      NUMBER(3) NOT NULL,
    descricao          VARCHAR2(200) NOT NULL,
    criacao            TIMESTAMP NOT NULL,
    prazo              TIMESTAMP NOT NULL,
    conclusao          TIMESTAMP,
    pet_id_pet         NUMBER(3) NOT NULL,
    status_id_status   NUMBER(3) NOT NULL,
    usuario_id_usuario NUMBER(3) NOT NULL,
    CONSTRAINT tarefa_pk PRIMARY KEY (id_tarefa),
    CONSTRAINT ck_tarefa_pontos CHECK (pontos_tarefa BETWEEN 0 AND 10),
    CONSTRAINT tarefa_pet_fk FOREIGN KEY (pet_id_pet) REFERENCES pet(id_pet),
    CONSTRAINT tarefa_status_fk FOREIGN KEY (status_id_status) REFERENCES
status(id_status),
    CONSTRAINT tarefa_usuario_fk FOREIGN KEY (usuario_id_usuario) REFERENCES
usuario(id_usuario)
);

-- 12. TRILHA (Educação e Adestramento)
CREATE TABLE trilha (
    id_trilha  NUMBER(5) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT NULL,
    nome       VARCHAR2(30) NOT NULL,
```

```
    descricao  VARCHAR2(200) NOT NULL,
    pet_id_pet NUMBER(3) NOT NULL,
    CONSTRAINT trilha_pk PRIMARY KEY (id_trilha),
    CONSTRAINT trilha_pet_fk FOREIGN KEY (pet_id_pet) REFERENCES pet(id_pet)
);

-- 13. MODULO
CREATE TABLE modulo (
    id_modulo        NUMBER(5) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT
NULL,
    nome             VARCHAR2(50) NOT NULL,
    tempo_conclusao  VARCHAR2(10) NOT NULL,
    descricao        VARCHAR2(100) NOT NULL,
    trilha_id_trilha NUMBER(5) NOT NULL,
    CONSTRAINT modulo_pk PRIMARY KEY (id_modulo),
    CONSTRAINT modulo_trilha_fk FOREIGN KEY (trilha_id_trilha) REFERENCES
trilha(id_trilha)
);

-- 14. AULA (Conteúdo Pedagógico desacoplado para MongoDB NoSQL)
CREATE TABLE aula (
    id_aula          NUMBER(5) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT
NULL,
    nome             VARCHAR2(50) NOT NULL,
    descricao        VARCHAR2(100) NOT NULL,
    pontos_aula      NUMBER(5) NOT NULL,
    dificuldade      VARCHAR2(20) NOT NULL,
    concluida        NUMBER NOT NULL,
    modulo_id_modulo NUMBER(5) NOT NULL,
    CONSTRAINT aula_pk PRIMARY KEY (id_aula),
    CONSTRAINT ck_aula_dificuldade CHECK (dificuldade IN ('AVANCADO', 'FACIL',
'MEDIO')),
    CONSTRAINT ck_aula_concluida CHECK (concluida IN (0, 1)),
    CONSTRAINT aula_modulo_fk FOREIGN KEY (modulo_id_modulo) REFERENCES
modulo(id_modulo)
);

-- 15. HISTORICO (Eventos Clínicos e de Bem-Estar do Pet)
CREATE TABLE historico (
    id_hist    NUMBER(3) GENERATED BY DEFAULT ON NULL AS IDENTITY NOT NULL,
    tipo_hist  VARCHAR2(30) NOT NULL,
    data_hist  TIMESTAMP NOT NULL,
    pet_id_pet NUMBER(3) NOT NULL,
    CONSTRAINT historico_pk PRIMARY KEY (id_hist),
    CONSTRAINT historico_pet_fk FOREIGN KEY (pet_id_pet) REFERENCES pet(id_pet)
);

-- TABELA DE AUDITORIA DML
```

```sql
CREATE TABLE auditoria_dml_tarefa (
    id_auditoria       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome_usuario       VARCHAR2(50) NOT NULL,
    tipo_operacao      VARCHAR2(10) NOT NULL,
    data_hora_operacao TIMESTAMP DEFAULT SYSTIMESTAMP NOT NULL,
    valores_anteriores VARCHAR2(4000),
    valores_novos      VARCHAR2(4000)
);

-- ÍNDICES EM CHAVES ESTRANGEIRAS (FKs) PARA PERFORMANCE E PREVENÇÃO DE TABLE
LOCKS
CREATE INDEX idx_cidade_estado ON cidade(estado_id_estado);
CREATE INDEX idx_bairro_cidade ON bairro(cidade_id_cidade);
CREATE INDEX idx_endereco_bairro ON endereco(bairro_id_bairro);
CREATE INDEX idx_usuario_endereco ON usuario(endereco_id_endereco);
CREATE INDEX idx_pet_raca ON pet(raca_id_raca);
CREATE INDEX idx_usuario_pet_pet ON usuario_pet(pet_id_pet);
CREATE INDEX idx_tarefa_pet ON tarefa(pet_id_pet);
CREATE INDEX idx_tarefa_status ON tarefa(status_id_status);
CREATE INDEX idx_tarefa_usuario ON tarefa(usuario_id_usuario);
CREATE INDEX idx_tarefa_status_prazo ON tarefa(status_id_status, prazo);
CREATE INDEX idx_trilha_pet ON trilha(pet_id_pet);
CREATE INDEX idx_modulo_trilha ON modulo(trilha_id_trilha);
CREATE INDEX idx_aula_modulo ON aula(modulo_id_modulo);
CREATE INDEX idx_historico_pet ON historico(pet_id_pet);

-- ÍNDICE CONDICIONAL: Garante no máximo 1 tutor responsável principal ativo
por pet no Care Circle
CREATE UNIQUE INDEX uq_usuario_pet_principal ON usuario_pet (CASE WHEN
respon_princ = 1 THEN pet_id_pet END);


-- PARTE 2: CARGA DE DADOS VÁLIDOS

-- 1. TELEFONE (5+ registros)
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (1, '11',
'981234567');
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (2, '11',
'972345678');
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (3, '11',
'963456789');
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (4, '11',
'954567890');
INSERT INTO telefone (id_telefone, num_ddd, num_tel) VALUES (5, '11',
'945678901');

-- 2. ESTADO (5+ registros)
INSERT INTO estado (id_estado, nome_estado) VALUES (1, 'São Paulo');
```

```sql
INSERT INTO estado (id_estado, nome_estado) VALUES (2, 'Rio de Janeiro');
INSERT INTO estado (id_estado, nome_estado) VALUES (3, 'Minas Gerais');
INSERT INTO estado (id_estado, nome_estado) VALUES (4, 'Paraná');
INSERT INTO estado (id_estado, nome_estado) VALUES (5, 'Santa Catarina');

-- 3. CIDADE (5+ registros)
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (1, 'São
Paulo', 1);
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (2,
'Campinas', 1);
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (3, 'Rio
de Janeiro', 2);
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (4, 'Belo
Horizonte', 3);
INSERT INTO cidade (id_cidade, nome_cidade, estado_id_estado) VALUES (5,
'Curitiba', 4);

-- 4. BAIRRO (5+ registros)
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (1, 'Bela
Vista', 1);
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (2,
'Pinheiros', 1);
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (3,
'Cambuí', 2);
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (4,
'Copacabana', 3);
INSERT INTO bairro (id_bairro, nome_bairro, cidade_id_cidade) VALUES (5,
'Savassi', 4);

-- 5. ENDERECO (5+ registros)
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES
(1, '01311000', 'Avenida Paulista', '1000', 1);
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES
(2, '05422000', 'Rua dos Pinheiros', '250', 2);
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES
(3, '13024000', 'Rua Coronel Silva Telles', '85', 3);
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES
(4, '22070010', 'Avenida Atlântica', '1500', 4);
INSERT INTO endereco (id_endereco, cep, rua, numero, bairro_id_bairro) VALUES
(5, '30140010', 'Avenida Getúlio Vargas', '300', 5);

-- 6. USUARIO (5+ registros com FK endereco_id_endereco e senhas em hash
BCrypt)
-- Senha de acesso para todos os usuários iniciais: 'Senha#123'
INSERT INTO usuario (id_usuario, nome, email, senha, role,
telefone_id_telefone, endereco_id_endereco)
```

```
VALUES (1, 'Carlos Silva', 'carlos.silva@email.com',
'$2a$10$nFdyYsVy8uiDRBO5KBpBWu9RKUQsIAM36oG9sK3ABRpcGike3MLv.', 'PREMIUM', 1,
1);
INSERT INTO usuario (id_usuario, nome, email, senha, role,
telefone_id_telefone, endereco_id_endereco)
VALUES (2, 'Mariana Oliveira', 'mariana.o@email.com',
'$2a$10$nFdyYsVy8uiDRBO5KBpBWu9RKUQsIAM36oG9sK3ABRpcGike3MLv.', 'COMUM', 2, 2);
INSERT INTO usuario (id_usuario, nome, email, senha, role,
telefone_id_telefone, endereco_id_endereco)
VALUES (3, 'Roberto Santos', 'roberto.s@email.com',
'$2a$10$nFdyYsVy8uiDRBO5KBpBWu9RKUQsIAM36oG9sK3ABRpcGike3MLv.', 'ADMIN', 3, 3);
INSERT INTO usuario (id_usuario, nome, email, senha, role,
telefone_id_telefone, endereco_id_endereco)
VALUES (4, 'Fernanda Costa', 'fernanda.c@email.com',
'$2a$10$nFdyYsVy8uiDRBO5KBpBWu9RKUQsIAM36oG9sK3ABRpcGike3MLv.', 'COMUM', 4, 4);
INSERT INTO usuario (id_usuario, nome, email, senha, role,
telefone_id_telefone, endereco_id_endereco)
VALUES (5, 'Lucas Mendes', 'lucas.m@email.com',
'$2a$10$nFdyYsVy8uiDRBO5KBpBWu9RKUQsIAM36oG9sK3ABRpcGike3MLv.', 'PREMIUM', 5,
5);

-- 7. RACA (5+ registros)
INSERT INTO raca (id_raca, nome_raca) VALUES (1, 'Vira-lata (SRD)');
INSERT INTO raca (id_raca, nome_raca) VALUES (2, 'Golden Retriever');
INSERT INTO raca (id_raca, nome_raca) VALUES (3, 'Pug');
INSERT INTO raca (id_raca, nome_raca) VALUES (4, 'Gato Persa');
INSERT INTO raca (id_raca, nome_raca) VALUES (5, 'Gato Siamês');

-- 8. PET (5+ registros)
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
VALUES (1, 'Thor', DATE '2021-04-10', 'M', 'GRANDE', 1, 2);
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
VALUES (2, 'Mel', DATE '2019-08-15', 'F', 'MEDIO', 1, 1);
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
VALUES (3, 'Bob', DATE '2022-01-20', 'M', 'PEQUENO', 0, 3);
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
VALUES (4, 'Mimi', DATE '2020-11-05', 'F', 'PEQUENO', 1, 4);
INSERT INTO pet (id_pet, nome, data_nasc, sexo, porte, castrado, raca_id_raca)
VALUES (5, 'Luna', DATE '2023-06-12', 'F', 'PEQUENO', 1, 5);

-- 9. USUARIO_PET (Care Circle familiar - 6 registros demonstrando cotutela)
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES
(1, 1, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES
(1, 2, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES
(2, 3, 1);
```

```sql
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES
(3, 4, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES
(4, 5, 1);
INSERT INTO usuario_pet (usuario_id_usuario, pet_id_pet, respon_princ) VALUES
(5, 1, 0);

-- 10. STATUS (Domínio Fechado / Finite State Machine)
INSERT INTO status (id_status, nome_status) VALUES (1, 'PENDENTE');
INSERT INTO status (id_status, nome_status) VALUES (2, 'CONCLUIDO');
INSERT INTO status (id_status, nome_status) VALUES (3, 'EXPIRADO');

-- 11. TAREFA (8 registros com variação de Pet e Status)
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao,
prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (1, 'Passeio Matinal', 5, 'Caminhada de 40 minutos no parque',
SYSTIMESTAMP - INTERVAL '2' DAY, SYSTIMESTAMP - INTERVAL '1' DAY, SYSTIMESTAMP
- INTERVAL '1' DAY, 1, 2, 1);

INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao,
prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (2, 'Medicamento Antibiótico', 10, 'Administrar 1 comprimido
pós-refeição', SYSTIMESTAMP - INTERVAL '1' DAY, SYSTIMESTAMP + INTERVAL '1'
DAY, NULL, 1, 1, 1);

INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao,
prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (3, 'Escovação de Pelos', 3, 'Remoção de subpelos mortos com
rasqueadeira', SYSTIMESTAMP - INTERVAL '3' DAY, SYSTIMESTAMP - INTERVAL '2'
DAY, NULL, 1, 3, 5);

INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao,
prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (4, 'Alimentação Especial', 4, 'Ração sênior com suplementação
vitamínica', SYSTIMESTAMP - INTERVAL '1' DAY, SYSTIMESTAMP + INTERVAL '1' DAY,
NULL, 2, 1, 1);

INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao,
prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (5, 'Banho e Higiene Bucal', 8, 'Banho morno com xampu neutro e
escovação dental', SYSTIMESTAMP - INTERVAL '4' DAY, SYSTIMESTAMP - INTERVAL '3'
DAY, SYSTIMESTAMP - INTERVAL '3' DAY, 2, 2, 1);

INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao,
prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (6, 'Corte de Unhas', 5, 'Aparar as pontas das unhas com alicate
apropriado', SYSTIMESTAMP - INTERVAL '2' DAY, SYSTIMESTAMP + INTERVAL '2' DAY,
NULL, 3, 1, 2);
```

```
INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao,
prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (7, 'Limpeza da Caixa de Areia', 4, 'Higienização completa e troca da
sílica', SYSTIMESTAMP - INTERVAL '2' DAY, SYSTIMESTAMP - INTERVAL '1' DAY,
SYSTIMESTAMP - INTERVAL '1' DAY, 4, 2, 3);

INSERT INTO tarefa (id_tarefa, titulo, pontos_tarefa, descricao, criacao,
prazo, conclusao, pet_id_pet, status_id_status, usuario_id_usuario)
VALUES (8, 'Estímulo com Varinha', 3, 'Sessão de brincadeiras de caça por 20
minutos', SYSTIMESTAMP - INTERVAL '1' DAY, SYSTIMESTAMP + INTERVAL '1' DAY,
NULL, 5, 1, 4);

-- 12. TRILHA (5+ registros)
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (1, 'Boas
Maneiras Caninas', 'Comandos básicos de obediência e disciplina', 1);
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (2,
'Socialização Filhotes', 'Introdução segura a outros animais e ambientes', 2);
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (3,
'Controle de Ansiedade', 'Redução do estresse de separação em cães de guarda',
3);
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (4,
'Enriquecimento Felino', 'Gatificação e estímulos sensoriais verticais', 4);
INSERT INTO trilha (id_trilha, nome, descricao, pet_id_pet) VALUES (5,
'Adaptação de Ambientes', 'Criação de rotinas calmas e seguras para gatos', 5);

-- 13. MODULO (5+ registros)
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao,
trilha_id_trilha) VALUES (1, 'Comando Senta e Fica', '45 min', 'Técnicas de
reforço positivo com petiscos', 1);
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao,
trilha_id_trilha) VALUES (2, 'Passeio Sem Puxar a Guia', '60 min',
'Equipamentos adequados e condução suave', 1);
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao,
trilha_id_trilha) VALUES (3, 'Apresentação a Novos Cães', '30 min', 'Leitura da
linguagem corporal canina', 2);
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao,
trilha_id_trilha) VALUES (4, 'Brinquedos Recheáveis', '40 min', 'Uso de kong e
tapetes de lamber para relaxamento', 3);
INSERT INTO modulo (id_modulo, nome, tempo_conclusao, descricao,
trilha_id_trilha) VALUES (5, 'Instalação de Nichos', '50 min', 'Posicionamento
estratégico de prateleiras', 4);

-- 14. AULA (6 registros sincronizados com MongoDB NoSQL - sem coluna
redundante de conteudo)
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade,
concluida, modulo_id_modulo)
```

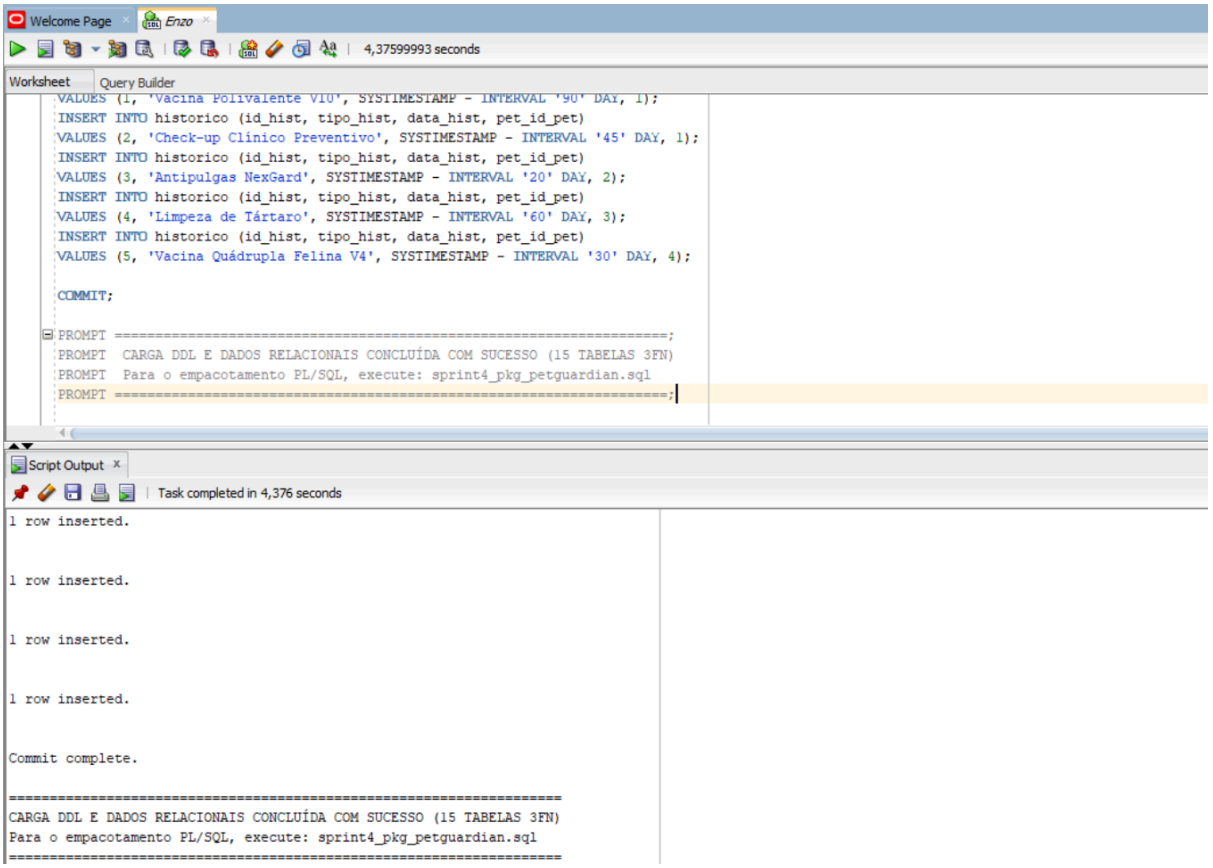
```
VALUES (1, 'Introdução ao Clicker', 'Fundamentos de marcação positiva', 50,
'FACIL', 1, 1);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade,
concluida, modulo_id_modulo)
VALUES (2, 'Critério de Duração do Fica', 'Aumentando o tempo gradativamente',
80, 'MEDIO', 0, 1);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade,
concluida, modulo_id_modulo)
VALUES (3, 'Troca de Foco na Guia', 'Atenção ao tutor em vez de estímulos da
rua', 70, 'MEDIO', 1, 2);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade,
concluida, modulo_id_modulo)
VALUES (4, 'Receitas de Enriquecimento', 'Preparo de petiscos congelados
saudáveis', 40, 'FACIL', 1, 4);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade,
concluida, modulo_id_modulo)
VALUES (5, 'Rotas de Fuga e Segurança', 'Evitando que o felino se sinta
acuado', 60, 'AVANCADO', 0, 5);
INSERT INTO aula (id_aula, nome, descricao, pontos_aula, dificuldade,
concluida, modulo_id_modulo)
VALUES (6, 'Estímulo Olfativo com Catnip e Silvervine', 'Enriquecimento
olfativo para felinos domésticos', 50, 'FACIL', 1, 5);

-- 15. HISTORICO (5+ registros)
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (1, 'Vacina Polivalente V10', SYSTIMESTAMP - INTERVAL '90' DAY, 1);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (2, 'Check-up Clínico Preventivo', SYSTIMESTAMP - INTERVAL '45' DAY, 1);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (3, 'Antipulgas NexGard', SYSTIMESTAMP - INTERVAL '20' DAY, 2);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (4, 'Limpeza de Tártaro', SYSTIMESTAMP - INTERVAL '60' DAY, 3);
INSERT INTO historico (id_hist, tipo_hist, data_hist, pet_id_pet)
VALUES (5, 'Vacina Quádrupla Felina V4', SYSTIMESTAMP - INTERVAL '30' DAY, 4);

COMMIT;

PROMPT ========================================================================;
PROMPT  CARGA DDL E DADOS RELACIONAIS CONCLUÍDA COM SUCESSO (15 TABELAS 3FN)
PROMPT  Para o empacotamento PL/SQL, execute: sprint4_pkg_petguardian.sql
PROMPT ========================================================================;
```

DML - Cargas de Dados



Objetos PL/SQL

Bloco 1: Objetos PL/SQL — Empacotamento (PKG_PETGUARDIAN) e Trigger

Empacotamento de Processos: Package PKG_PETGUARDIAN

Na Sprint 4, todas as regras procedurais de negócio do banco relacional foram encapsuladas no pacote corporativo **PKG_PETGUARDIAN**, dividido em **Especificação (PACKAGE)** e **Corpo (PACKAGE BODY)**, garantindo modularidade, segurança transacional e integração direta com o backend:

Subprograma	**Tipo**	**Finalidade de Negócio / Consumo na Aplicação**
pr_calcular_pontuacao_pet	Procedure	Calcula os pontos consolidados de tarefas concluídas e aulas concluídas de um pet específico (GET /pets/{id}/pontos).
pr_calcular_pontuacao_usuario	Procedure	Calcula a pontuação agregada de todos os pets sob tutela do usuário (via USUARIO_PET) e retorna, em lote único sem consultas N+1, o detalhamento individual de cada pet serializado em JSON nativo Oracle 19c (JSON_ARRAYAGG + JSON_OBJECT via p_detalhe_pets_json OUT CLOB). Consumo Java/App: GET /pets/me/pontos.
pr_obter_resumo_rede_cuidado	Procedure	Consolida em chamada única as métricas do Care Circle do tutor: total de pets, co-cuidadores, tarefas pendentes, concluídas e XP acumulado (GET /usuarios/me/rede-cuidado).
fn_obter_nivel_usuario	Function	Classifica o nível de gamificação do tutor (BRONZE: 0–50 pts, PRATA: 51–150 pts, OURO: 151+ pts) com base no XP acumulado (GET /usuarios/me).
pr_transferir_responsavel_principal	Procedure	Transfere atomicamente a titularidade principal (respon_princ = 1) de um pet para outro membro do Care Circle em USUARIO_PET (PATCH /pets/{id}/responsavel-principal).

Subprograma	Tipo	Finalidade de Negócio / Consumo na Aplicação
pr_registrar_auditoria_tarefa	Procedure	Grava de forma modularizada o histórico de modificações da tabela TAREFA diretamente na tabela AUDITORIA_DML_TAREFA.
pr_exportar_aulas_json	Procedure	Executa a extração analítica nativa dos registros da tabela relacional AULA em formato JSON serializado (CLOB) via JSON_ARRAYAGG + JSON_OBJECT, alimentando o pipeline NoSQL. (Consumo: Exportação automatizada de dados para o MongoDB)
trg_audit_tarefa	Trigger DML	Disparada automaticamente (AFTER INSERT OR UPDATE OR DELETE FOR EACH ROW) na tabela TAREFA, gravando usuário, operação, data/hora e estados :OLD e :NEW em AUDITORIA_DML_TAREFA.

Exceções Personalizadas Tratadas no Pacote

Exceção PL/SQL	Código ORA	Condição de Disparo
e_id_invalido	-20018	ID do pet ou do usuário informado é nulo ou <= 0.
e_pet_inexistente	-20019	Pet não encontrado na tabela PET.
OTHERS (Score Pet)	-20020	Erro inesperado ao consolidar pontuação do pet.
e_usuario_inexistente	-20022	Usuário não encontrado na tabela USUARIO.
OTHERS (Score Usuário)	-20023	Erro inesperado ao consolidar pontuação do usuário.
OTHERS (Rede Cuidado)	-20024	Erro inesperado ao gerar resumo da rede de cuidado.
OTHERS (Nível Usuário)	-20025	Erro inesperado ao classificar o nível de gamificação do usuário (fn_obter_nivel_usuario).
e_vinculo_inexistente	-20041	Novo responsável não possui vínculo prévio com o pet em USUARIO_PET.

Exceção PL/SQL	Código ORA	Condição de Disparo
OTHERS (Transferência)	-20042	Erro inesperado ao transferir responsabilidade principal.
OTHERS (Auditoria)	-20043	Erro ao registrar auditoria de tarefa.
OTHERS (Exportação JSON)	-20044	Falha durante a extração e serialização do dataset JSON de aulas.

Código SQL Completo (sprint4_pkg_petguardian.sql)

```sql
SET SERVEROUTPUT ON SIZE UNLIMITED;

-- ========================================================================
-- PARTE 1: ESPECIFICAÇÃO DO PACOTE (PACKAGE SPECIFICATION)
-- ========================================================================
CREATE OR REPLACE PACKAGE pkg_petguardian AS

    -- Exceções públicas com códigos ORA padronizados
    e_id_invalido           EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_id_invalido, -20018);

    e_pet_inexistente       EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_pet_inexistente, -20019);

    e_usuario_inexistente   EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_usuario_inexistente, -20022);

    e_vinculo_inexistente   EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_vinculo_inexistente, -20041);

    -- ------------------------------------------------------------------
    -- SEÇÃO 1: PONTUAÇÃO GAMIFICADA DE PETS E USUÁRIOS (APP & JAVA)
    -- ARQUITETURA PET-CENTRIC: A pontuação (XP) pertence ao PET.
    -- Todos os co-cuidadores compartilham e visualizam a mesma evolução.
    -- ------------------------------------------------------------------

    -- 1.1 Cálculo consolidado de pontuação do pet individual (tarefas + aulas)
    PROCEDURE pr_calcular_pontuacao_pet (
        p_id_pet         IN NUMBER,
        p_pontos_tarefas OUT NUMBER,
        p_pontos_aulas   OUT NUMBER,
        p_pontos_totais  OUT NUMBER
    );
```

```
    -- 1.2 Cálculo agregado de todos os pets do usuário (com detalhamento em
JSON nativo)
    PROCEDURE pr_calcular_pontuacao_usuario (
        p_id_usuario        IN NUMBER,
        p_pontos_tarefas    OUT NUMBER,
        p_pontos_aulas      OUT NUMBER,
        p_pontos_totais     OUT NUMBER,
        p_detalhe_pets_json OUT CLOB
    );

    -- 1.3 Relatório consolidado da rede de cuidado do usuário
(/usuarios/me/rede-cuidado)
    PROCEDURE pr_obter_resumo_rede_cuidado (
        p_id_usuario         IN NUMBER,
        p_total_pets         OUT NUMBER,
        p_total_cuidadores   OUT NUMBER,
        p_tarefas_pendentes  OUT NUMBER,
        p_tarefas_concluidas OUT NUMBER,
        p_pontos_acumulados  OUT NUMBER
    );

    -- 1.4 Classificação de engajamento do tutor baseada no XP consolidado dos
seus pets
    FUNCTION fn_obter_nivel_usuario (
        p_id_usuario IN NUMBER
    ) RETURN VARCHAR2;

    -- ---------------------------------------------------------------------
    -- SEÇÃO 2: OPERAÇÕES TRANSACIONAIS, AUDITORIA E EXPORTAÇÃO JSON
    -- ---------------------------------------------------------------------

    -- 2.1 Transferência atômica de responsabilidade principal no Care Circle
(USUARIO_PET)
    PROCEDURE pr_transferir_responsavel_principal (
        p_id_pet          IN NUMBER,
        p_novo_id_usuario IN NUMBER
    );

    -- 2.2 Gravação modular de auditoria DML (invocado pela trigger
trg_audit_tarefa)
    PROCEDURE pr_registrar_auditoria_tarefa (
        p_tipo_operacao  IN VARCHAR2,
        p_old_id_tarefa  IN NUMBER,
        p_old_titulo     IN VARCHAR2,
        p_old_pontos     IN NUMBER,
        p_old_status     IN NUMBER,
        p_old_pet        IN NUMBER,
        p_old_usuario    IN NUMBER,
```

```
        p_new_id_tarefa  IN NUMBER,
        p_new_titulo     IN VARCHAR2,
        p_new_pontos     IN NUMBER,
        p_new_status     IN NUMBER,
        p_new_pet        IN NUMBER,
        p_new_usuario    IN NUMBER
    );

    -- 2.3 Exportação nativa do dataset relacional de AULA em formato JSON
    PROCEDURE pr_exportar_aulas_json (
        p_json_dataset OUT CLOB
    );

END pkg_petguardian;
/

-- ======================================================================
-- PARTE 2: CORPO DO PACOTE (PACKAGE BODY)
-- ======================================================================
CREATE OR REPLACE PACKAGE BODY pkg_petguardian AS

    -- ----------------------------------------------------------------
    -- SEÇÃO 1: PONTUAÇÃO GAMIFICADA DE PETS E USUÁRIOS
    -- ----------------------------------------------------------------

    -- 1.1 Cálculo consolidado de pontuação do pet individual (tarefas + aulas)
    -- Os pontos pertencem ao Pet: tarefas e aulas realizadas somam ao XP do
animal
    PROCEDURE pr_calcular_pontuacao_pet (
        p_id_pet         IN NUMBER,
        p_pontos_tarefas OUT NUMBER,
        p_pontos_aulas   OUT NUMBER,
        p_pontos_totais  OUT NUMBER
    ) IS
        v_existe NUMBER;
    BEGIN
        IF p_id_pet IS NULL OR p_id_pet <= 0 THEN
            RAISE_APPLICATION_ERROR(-20018, 'ID do pet fornecido é nulo ou
inválido.');
        END IF;

        SELECT COUNT(1) INTO v_existe FROM pet WHERE id_pet = p_id_pet;
        IF v_existe = 0 THEN
            RAISE_APPLICATION_ERROR(-20019, 'Pet com id ' || p_id_pet || ' não
foi encontrado.');
        END IF;
```

```
        -- Soma de tarefas concluídas do pet (busca dinâmica por status
'CONCLUIDO', sem ID mágico)
        SELECT NVL(SUM(t.pontos_tarefa), 0)
          INTO p_pontos_tarefas
          FROM tarefa t
          JOIN status s ON t.status_id_status = s.id_status
         WHERE t.pet_id_pet = p_id_pet
           AND s.nome_status = 'CONCLUIDO';

        -- Soma de aulas concluídas associadas às trilhas e módulos do pet
        SELECT NVL(SUM(a.pontos_aula), 0)
          INTO p_pontos_aulas
          FROM aula a
          JOIN modulo m ON a.modulo_id_modulo = m.id_modulo
          JOIN trilha tr ON m.trilha_id_trilha = tr.id_trilha
         WHERE tr.pet_id_pet = p_id_pet
           AND a.concluida = 1;

        p_pontos_totais := p_pontos_tarefas + p_pontos_aulas;
    EXCEPTION
        WHEN e_id_invalido OR e_pet_inexistente THEN
            RAISE;
        WHEN OTHERS THEN
            RAISE_APPLICATION_ERROR(-20020, 'Erro ao calcular pontuação
consolidada do pet: ' || SQLERRM);
    END pr_calcular_pontuacao_pet;

    -- 1.2 Cálculo agregado de todos os pets do usuário (com detalhamento em
JSON nativo)
    PROCEDURE pr_calcular_pontuacao_usuario (
        p_id_usuario        IN NUMBER,
        p_pontos_tarefas    OUT NUMBER,
        p_pontos_aulas      OUT NUMBER,
        p_pontos_totais     OUT NUMBER,
        p_detalhe_pets_json OUT CLOB
    ) IS
        v_existe NUMBER;
    BEGIN
        IF p_id_usuario IS NULL OR p_id_usuario <= 0 THEN
            RAISE_APPLICATION_ERROR(-20018, 'ID do usuário fornecido é nulo ou
inválido.');
        END IF;

        SELECT 1 INTO v_existe FROM usuario WHERE id_usuario = p_id_usuario;

        WITH pet_pontos AS (
            SELECT p.id_pet, p.nome,
```

```
                    NVL((SELECT SUM(t.pontos_tarefa) FROM tarefa t JOIN status s
ON t.status_id_status = s.id_status WHERE t.pet_id_pet = p.id_pet AND
s.nome_status = 'CONCLUIDO'), 0) tar,
                    NVL((SELECT SUM(a.pontos_aula) FROM aula a JOIN modulo m ON
a.modulo_id_modulo = m.id_modulo JOIN trilha tr ON m.trilha_id_trilha =
tr.id_trilha WHERE tr.pet_id_pet = p.id_pet AND a.concluida = 1), 0) aul
                FROM pet p JOIN usuario_pet up ON p.id_pet = up.pet_id_pet
               WHERE up.usuario_id_usuario = p_id_usuario
        )
        SELECT NVL(SUM(tar), 0), NVL(SUM(aul), 0),
               NVL(JSON_ARRAYAGG(
                   JSON_OBJECT(
                       'petId'         VALUE id_pet,
                       'nomePet'       VALUE nome,
                       'pontosTarefas' VALUE tar,
                       'pontosAulas'   VALUE aul,
                       'pontosTotais'  VALUE (tar + aul)
                   ) ORDER BY id_pet RETURNING CLOB
               ), '[]')
          INTO p_pontos_tarefas, p_pontos_aulas, p_detalhe_pets_json
          FROM pet_pontos;

        p_pontos_totais := p_pontos_tarefas + p_pontos_aulas;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20022, 'Usuário com id ' || p_id_usuario
|| ' não foi encontrado.');
        WHEN e_id_invalido OR e_usuario_inexistente THEN
            RAISE;
        WHEN OTHERS THEN
            RAISE_APPLICATION_ERROR(-20023, 'Erro ao calcular pontuação
agregada do usuário: ' || SQLERRM);
    END pr_calcular_pontuacao_usuario;

    -- 1.3 Relatório consolidado da rede de cuidado do usuário
(/usuarios/me/rede-cuidado)
    PROCEDURE pr_obter_resumo_rede_cuidado (
        p_id_usuario         IN NUMBER,
        p_total_pets         OUT NUMBER,
        p_total_cuidadores   OUT NUMBER,
        p_tarefas_pendentes  OUT NUMBER,
        p_tarefas_concluidas OUT NUMBER,
        p_pontos_acumulados  OUT NUMBER
    ) IS
        v_existe        NUMBER;
        v_total_tarefas NUMBER;
        v_total_aulas   NUMBER;
        v_json_pets     CLOB;
```

```
    BEGIN
        IF p_id_usuario IS NULL OR p_id_usuario <= 0 THEN
            RAISE_APPLICATION_ERROR(-20018, 'ID do usuário fornecido é nulo ou
inválido.');
        END IF;

        SELECT COUNT(1) INTO v_existe FROM usuario WHERE id_usuario =
p_id_usuario;
        IF v_existe = 0 THEN
            RAISE_APPLICATION_ERROR(-20022, 'Usuário com id ' || p_id_usuario
|| ' não foi encontrado.');
        END IF;

        -- Total de pets tutelados ou co-tutelados pelo usuário
        SELECT NVL(COUNT(DISTINCT up.pet_id_pet), 0)
          INTO p_total_pets
          FROM usuario_pet up
         WHERE up.usuario_id_usuario = p_id_usuario;

        -- Total de co-cuidadores distintos na rede do tutor
        SELECT NVL(COUNT(DISTINCT up2.usuario_id_usuario), 0)
          INTO p_total_cuidadores
          FROM usuario_pet up2
         WHERE up2.usuario_id_usuario <> p_id_usuario
           AND up2.pet_id_pet IN (
               SELECT up.pet_id_pet
                 FROM usuario_pet up
                WHERE up.usuario_id_usuario = p_id_usuario
           );

        -- Tarefas pendentes e concluídas da rede do tutor (busca dinâmica por
status 'PENDENTE' e 'CONCLUIDO')
        SELECT NVL(COUNT(CASE WHEN s.nome_status = 'PENDENTE' THEN 1 END), 0),
               NVL(COUNT(CASE WHEN s.nome_status = 'CONCLUIDO' THEN 1 END), 0)
          INTO p_tarefas_pendentes,
               p_tarefas_concluidas
          FROM tarefa t
          JOIN status s ON t.status_id_status = s.id_status
         WHERE t.pet_id_pet IN (
             SELECT up.pet_id_pet
               FROM usuario_pet up
              WHERE up.usuario_id_usuario = p_id_usuario
         );

        -- Pontuação acumulada total somada (tarefas concluídas + aulas
finalizadas)
        pr_calcular_pontuacao_usuario(p_id_usuario, v_total_tarefas,
v_total_aulas, p_pontos_acumulados, v_json_pets);
```

```
    EXCEPTION
        WHEN e_id_invalido OR e_usuario_inexistente THEN
            RAISE;
        WHEN OTHERS THEN
            RAISE_APPLICATION_ERROR(-20024, 'Erro ao obter resumo da rede de
cuidado: ' || SQLERRM);
    END pr_obter_resumo_rede_cuidado;

    -- 1.4 Classificação de engajamento do tutor baseada no XP consolidado dos
seus pets
    FUNCTION fn_obter_nivel_usuario (
        p_id_usuario IN NUMBER
    ) RETURN VARCHAR2 IS
        v_tar       NUMBER;
        v_aul       NUMBER;
        v_tot       NUMBER;
        v_json_pets CLOB;
    BEGIN
        IF p_id_usuario IS NULL OR p_id_usuario <= 0 THEN
            RAISE_APPLICATION_ERROR(-20018, 'ID do usuário fornecido é nulo ou
inválido.');
        END IF;

        pr_calcular_pontuacao_usuario(p_id_usuario, v_tar, v_aul, v_tot,
v_json_pets);

        IF v_tot <= 50 THEN
            RETURN 'BRONZE';
        ELSIF v_tot <= 150 THEN
            RETURN 'PRATA';
        ELSE
            RETURN 'OURO';
        END IF;
    EXCEPTION
        WHEN e_id_invalido OR e_usuario_inexistente THEN
            RAISE;
        WHEN OTHERS THEN
            RAISE_APPLICATION_ERROR(-20025, 'Erro ao obter nível do usuário: '
|| SQLERRM);
    END fn_obter_nivel_usuario;

    -- ---------------------------------------------------------------
    -- SEÇÃO 2: OPERAÇÕES TRANSACIONAIS, AUDITORIA E EXPORTAÇÃO JSON
    -- ---------------------------------------------------------------

    -- 2.1 Transferência atômica de responsabilidade principal no Care Circle
(USUARIO_PET)
    PROCEDURE pr_transferir_responsavel_principal (
```

```
        p_id_pet          IN NUMBER,
        p_novo_id_usuario IN NUMBER
    ) IS
        v_existe_vinculo NUMBER;
    BEGIN
        IF p_id_pet IS NULL OR p_id_pet <= 0 OR p_novo_id_usuario IS NULL OR
p_novo_id_usuario <= 0 THEN
            RAISE_APPLICATION_ERROR(-20018, 'ID do pet e do novo responsável
não podem ser nulos ou inválidos.');
        END IF;

        -- Valida se o novo responsável já faz parte do Care Circle do pet
        SELECT COUNT(1) INTO v_existe_vinculo
          FROM usuario_pet up
         WHERE up.pet_id_pet = p_id_pet
           AND up.usuario_id_usuario = p_novo_id_usuario;

        IF v_existe_vinculo = 0 THEN
            RAISE_APPLICATION_ERROR(-20041, 'O novo responsável (id ' ||
p_novo_id_usuario || ') não está vinculado ao pet ' || p_id_pet || ' em
USUARIO_PET.');
        END IF;

        -- Transação atômica otimizada em uma única operação DML (evita duplo
lock e estado intermediário sem titular)
        UPDATE usuario_pet
           SET respon_princ = CASE
                                WHEN usuario_id_usuario = p_novo_id_usuario
THEN 1
                                ELSE 0
                              END
         WHERE pet_id_pet = p_id_pet;
    EXCEPTION
        WHEN e_id_invalido OR e_vinculo_inexistente THEN
            RAISE;
        WHEN OTHERS THEN
            RAISE_APPLICATION_ERROR(-20042, 'Erro ao transferir
responsabilidade principal: ' || SQLERRM);
    END pr_transferir_responsavel_principal;

    -- 2.2 Gravação modular de auditoria DML (invocado pela trigger
trg_audit_tarefa)
    PROCEDURE pr_registrar_auditoria_tarefa (
        p_tipo_operacao  IN VARCHAR2,
        p_old_id_tarefa  IN NUMBER,
        p_old_titulo     IN VARCHAR2,
        p_old_pontos     IN NUMBER,
        p_old_status     IN NUMBER,
```

```
        p_old_pet        IN NUMBER,
        p_old_usuario    IN NUMBER,
        p_new_id_tarefa  IN NUMBER,
        p_new_titulo     IN VARCHAR2,
        p_new_pontos     IN NUMBER,
        p_new_status     IN NUMBER,
        p_new_pet        IN NUMBER,
        p_new_usuario    IN NUMBER
    ) IS
        PRAGMA AUTONOMOUS_TRANSACTION;
        v_valores_old VARCHAR2(4000) := NULL;
        v_valores_new VARCHAR2(4000) := NULL;
    BEGIN
        IF p_tipo_operacao IN ('INSERT', 'UPDATE') THEN
            v_valores_new := 'ID_TAREFA=' || p_new_id_tarefa ||
                             '; TITULO=' || p_new_titulo ||
                             '; PONTOS=' || p_new_pontos ||
                             '; STATUS=' || p_new_status ||
                             '; PET=' || p_new_pet ||
                             '; USUARIO=' || p_new_usuario;
        END IF;

        IF p_tipo_operacao IN ('UPDATE', 'DELETE') THEN
            v_valores_old := 'ID_TAREFA=' || p_old_id_tarefa ||
                             '; TITULO=' || p_old_titulo ||
                             '; PONTOS=' || p_old_pontos ||
                             '; STATUS=' || p_old_status ||
                             '; PET=' || p_old_pet ||
                             '; USUARIO=' || p_old_usuario;
        END IF;

        INSERT INTO auditoria_dml_tarefa (
            nome_usuario, tipo_operacao, data_hora_operacao,
valores_anteriores, valores_novos
        ) VALUES (
            USER, p_tipo_operacao, SYSTIMESTAMP, v_valores_old, v_valores_new
        );
        COMMIT;
    EXCEPTION
        WHEN OTHERS THEN
            ROLLBACK;
            RAISE_APPLICATION_ERROR(-20043, 'Erro ao registrar auditoria de
tarefa: ' || SQLERRM);
    END pr_registrar_auditoria_tarefa;

    -- 2.3 Exportação nativa do dataset relacional de AULA em formato JSON
    PROCEDURE pr_exportar_aulas_json (
        p_json_dataset OUT CLOB
```

```
    ) IS
    BEGIN
        SELECT JSON_SERIALIZE(
            JSON_ARRAYAGG(
                JSON_OBJECT(
                    'id_aula'          VALUE a.id_aula,
                    'nome'             VALUE a.nome,
                    'descricao'        VALUE a.descricao,
                    'pontos_aula'      VALUE a.pontos_aula,
                    'dificuldade'      VALUE a.dificuldade,
                    'concluida'        VALUE a.concluida,
                    'modulo_id_modulo' VALUE a.modulo_id_modulo
                )
                ORDER BY a.id_aula
            ) RETURNING CLOB PRETTY
        )
        INTO p_json_dataset
        FROM aula a;
    EXCEPTION
        WHEN OTHERS THEN
            RAISE_APPLICATION_ERROR(-20044, 'Erro ao exportar dataset JSON de
aulas: ' || SQLERRM);
    END pr_exportar_aulas_json;

END pkg_petguardian;
/

-- ======================================================================
-- PARTE 3: TRIGGER DE AUDITORIA DML (AUDITORIA_DML_TAREFA)
-- DISPARADOR LEVE QUE DELEGA A EXECUÇÃO PARA A PACKAGE EMPACOTADA
-- ======================================================================
CREATE OR REPLACE TRIGGER trg_audit_tarefa
AFTER INSERT OR UPDATE OR DELETE ON tarefa
FOR EACH ROW
BEGIN
    pkg_petguardian.pr_registrar_auditoria_tarefa(
        p_tipo_operacao  => CASE WHEN INSERTING THEN 'INSERT' WHEN UPDATING
THEN 'UPDATE' ELSE 'DELETE' END,
        p_old_id_tarefa  => :OLD.id_tarefa,
        p_old_titulo     => :OLD.titulo,
        p_old_pontos     => :OLD.pontos_tarefa,
        p_old_status     => :OLD.status_id_status,
        p_old_pet        => :OLD.pet_id_pet,
        p_old_usuario    => :OLD.usuario_id_usuario,
        p_new_id_tarefa  => :NEW.id_tarefa,
        p_new_titulo     => :NEW.titulo,
        p_new_pontos     => :NEW.pontos_tarefa,
        p_new_status     => :NEW.status_id_status,
```

```
        p_new_pet        => :NEW.pet_id_pet,
        p_new_usuario    => :NEW.usuario_id_usuario
    );
END trg_audit_tarefa;
/

-- ======================================================================
-- PARTE 4: BATERIA DE TESTES DE INTEGRAÇÃO (PKG_PETGUARDIAN)
-- ======================================================================

-- 1. Teste de Pontuação do Pet Individual (Thor - ID 1)
DECLARE
    v_tar NUMBER;
    v_aul NUMBER;
    v_tot NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== TESTE 1: pr_calcular_pontuacao_pet ===');
    pkg_petguardian.pr_calcular_pontuacao_pet(1, v_tar, v_aul, v_tot);
    DBMS_OUTPUT.PUT_LINE('Thor (ID 1) -> Tarefas: ' || v_tar || ' | Aulas: ' ||
v_aul || ' | Total: ' || v_tot);
END;
/

-- 2. Teste de Pontuação Agregada e Nível do Tutor (Carlos Silva - ID 1)
DECLARE
    v_tar_usr  NUMBER;
    v_aul_usr  NUMBER;
    v_tot_usr  NUMBER;
    v_json_usr CLOB;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== TESTE 2: pr_calcular_pontuacao_usuario e
fn_obter_nivel_usuario ===');
    pkg_petguardian.pr_calcular_pontuacao_usuario(1, v_tar_usr, v_aul_usr,
v_tot_usr, v_json_usr);
    DBMS_OUTPUT.PUT_LINE('Carlos Silva (ID 1) -> Tarefas: ' || v_tar_usr || ' |
Aulas: ' || v_aul_usr || ' | Total: ' || v_tot_usr);
    DBMS_OUTPUT.PUT_LINE('Detalhe JSON dos pets: ' || v_json_usr);
    DBMS_OUTPUT.PUT_LINE('Nível de Gamificação do Tutor (FUNCTION): ' ||
pkg_petguardian.fn_obter_nivel_usuario(1));
END;
/

-- 3. Teste de Resumo da Rede de Cuidado (Carlos Silva - ID 1)
DECLARE
    v_tot_pets NUMBER;
    v_tot_cuid NUMBER;
    v_tar_pend NUMBER;
    v_tar_conc NUMBER;
```

```
    v_pts_acum NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== TESTE 3: pr_obter_resumo_rede_cuidado (Carlos
Silva ID 1) ===');
    pkg_petguardian.pr_obter_resumo_rede_cuidado(1, v_tot_pets, v_tot_cuid,
v_tar_pend, v_tar_conc, v_pts_acum);
    DBMS_OUTPUT.PUT_LINE('Carlos Silva (ID 1) -> Pets: ' || v_tot_pets ||
                         ' | Cuidadores: ' || v_tot_cuid ||
                         ' | Tarefas Pendentes: ' || v_tar_pend ||
                         ' | Tarefas Concluídas: ' || v_tar_conc ||
                         ' | Pontos Acumulados: ' || v_pts_acum);
END;
/

-- 4. Teste de Transferência de Titularidade no Care Circle
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== TESTE 4: pr_transferir_responsavel_principal
===');
    pkg_petguardian.pr_transferir_responsavel_principal(1, 5);
    DBMS_OUTPUT.PUT_LINE('Titularidade de Thor transferida com sucesso para
Lucas Mendes (ID 5).');
END;
/

-- 5. Disparo e Evidências da Trigger de Auditoria DML (TRG_AUDIT_TAREFA)
DECLARE
    v_id_concluido NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== TESTE 5: Disparo da Trigger trg_audit_tarefa na
Tabela TAREFA ===');
    SELECT id_status INTO v_id_concluido FROM status WHERE nome_status =
'CONCLUIDO';

    UPDATE tarefa
       SET status_id_status = v_id_concluido,
           conclusao = SYSTIMESTAMP
     WHERE id_tarefa = 2;
    DBMS_OUTPUT.PUT_LINE('Tarefa ID 2 atualizada com sucesso. Registro de
auditoria gerado pela trigger:');
END;
/

-- 6. Teste de Exportação do Dataset JSON de Aulas Embutido no Pacote
DECLARE
    v_dataset_json CLOB;
BEGIN
    DBMS_OUTPUT.PUT_LINE('=== TESTE 6: pr_exportar_aulas_json (Empacotamento do
Dataset JSON) ===');
```

```
    pkg_petguardian.pr_exportar_aulas_json(v_dataset_json);
    DBMS_OUTPUT.PUT_LINE('Dataset JSON exportado com sucesso (' ||
DBMS_LOB.GETLENGTH(v_dataset_json) || ' caracteres).');
END;
/

SELECT id_auditoria, nome_usuario, tipo_operacao, data_hora_operacao,
valores_anteriores, valores_novos
  FROM auditoria_dml_tarefa
 ORDER BY id_auditoria ASC;

-- Rollback das operações demonstrativas de teste para preservar o estado limpo
das sementes
ROLLBACK;
```

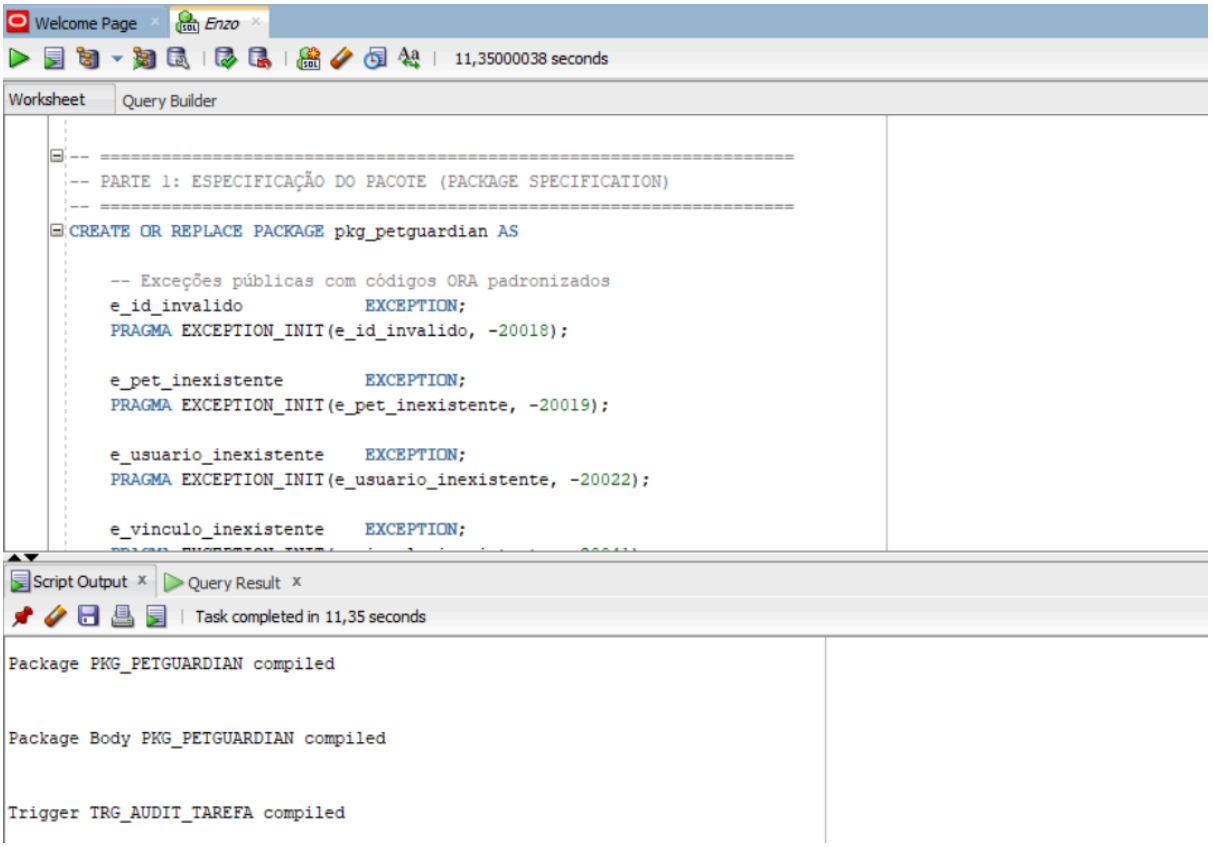
Bloco 2: Bateria de Testes e Evidências de Execução (PKG_PETGUARDIAN)

Resultados da Bateria de Testes Integrada (Testes 1 a 6)

Teste	Subprograma Avaliado	Entrada	Resultado Obtido (DBMS_OUTPUT)
Teste 1	pr_calcular_pontuacao_pet	p_id_pet = 1 (Thor)	Tarefas: 5 | Aulas: 120 | Total: 125
Teste 2	pr_calcular_pontuacao_usuario + fn_obter_nivel_usuario	p_id_usuario = 1 (Carlos Silva)	Tarefas: 13 | Aulas: 120 | Total: 133 Detalhe JSON: [{"petId":1,"nomePet":"Thor","pontosTarefas":5,"pontosAulas":120,"pontosTotais":125},{"petId":2,"nomePet":"Mel","pontosTarefas":8,"pontosAulas":0,"pontosTotais":8}] ➜ Nível: PRATA
Teste 3	pr_obter_resumo_rede_cuidado	p_id_usuario = 1 (Carlos Silva)	Pets: 2 | Cuidadores: 1 | Pendentes: 2 | Concluídas: 2 | Pontos: 133
Teste 4	pr_transferir_responsavel_principal	p_id_pet = 1, p_novo_id_usuario = 5	Titularidade de Thor transferida com sucesso para Lucas Mendes (ID 5).

Teste	Subprograma Avaliado	Entrada	Resultado Obtido (DBMS_OUTPUT)
Teste 5	trg_audit_tarefa (Trigger DML)	UPDATE tarefa SET status_id_status = (SELECT id_status FROM status WHERE nome_status = 'CONCLUIDO') WHERE id_tarefa = 2	Registro gravado na tabela AUDITORIA_DML_TAREFA com tipo_operacao = 'UPDATE' e campos :OLD e :NEW.
Teste 6	pr_exportar_aulas_json	Invocação direta da procedure do pacote	Dataset JSON exportado com sucesso (948 caracteres). Serialização analítica completa de todos os registros de AULA.

Evidências de Execução no Oracle SQL Developer



```
=== TESTE 1: pr_calcular_pontuacao_pet ===
Thor (ID 1) -> Tarefas: 5 | Aulas: 120 | Total: 125


PL/SQL procedure successfully completed.

=== TESTE 2: pr_calcular_pontuacao_usuario e fn_obter_nivel_usuario ===
Carlos Silva (ID 1) -> Tarefas: 13 | Aulas: 120 | Total: 133
Detalhe JSON dos pets: [{"petId":1,"nomePet":"Thor","pontosTarefas":5,"pontosAulas":120,"pontosTotais":125},{"petId":2,"nomePet":"Mel","pontosTarefas":8,"pontosAulas":0,"pontosTotais":8}]
Nivel de Gamificação do Tutor (FUNCTION): PRATA


PL/SQL procedure successfully completed.

=== TESTE 3: pr_obter_resumo_rede_cuidado (Carlos Silva ID 1) ===
Carlos Silva (ID 1) -> Pets: 2 | Cuidadores: 1 | Tarefas Pendentes: 2 | Tarefas Concluídas: 2 | Pontos Acumulados: 133


PL/SQL procedure successfully completed.

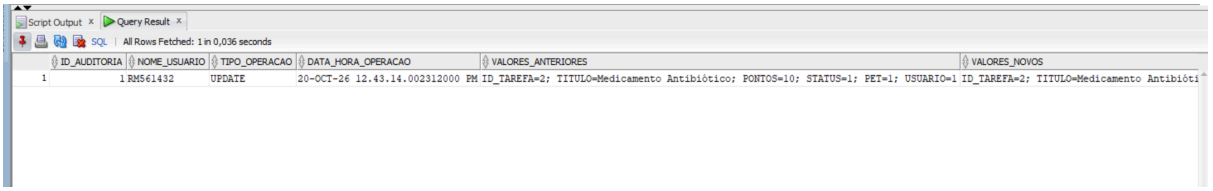
=== TESTE 4: pr_transferir_responsavel_principal ===
Titularidade de Thor transferida com sucesso para Lucas Mendes (ID 5).

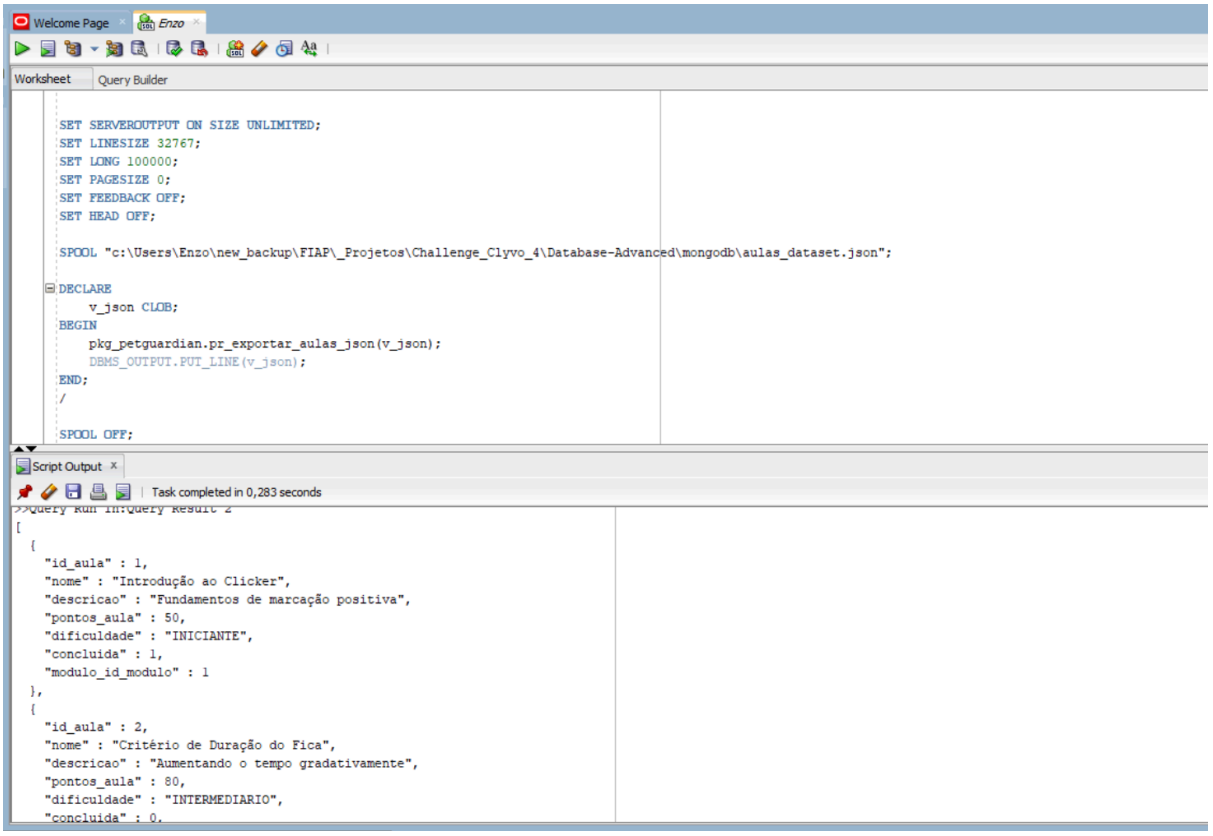

PL/SQL procedure successfully completed.

=== TESTE 5: Disparo da Trigger trg_audit_tarefa na Tabela TAREFA ===
Tarefa ID 2 atualizada com sucesso. Registro de auditoria gerado pela trigger:


PL/SQL procedure successfully completed.

=== TESTE 6: pr_exportar_aulas_json (Empacotamento do Dataset JSON) ===
Dataset JSON exportado com sucesso (1406 caracteres).
```





Bloco 3: Banco de Dados Não Relacional — MongoDB (aulas e conteudos_aula)

Persistência Poliglota e Arquitetura Dual de Coleções

Para atender às exigências de escalabilidade e desacoplamento da disciplina de Banco de Dados e apoiar a arquitetura do backend Java e do aplicativo Mobile, o banco NoSQL **petguardian** no MongoDB Atlas opera com **duas coleções especializadas**:

1. **Coleção aulas:** Armazena os documentos originados da exportação direta dos dados relacionais da tabela **AULA** do Oracle 19c. Preserva os atributos estruturais cadastrados (título, descrição, pontos da aula, nível de dificuldade, flag de conclusão e referência ao módulo/trilha).
2. **Coleção conteudos_aula:** Responsável pelos conteúdos educacionais ricos associados às aulas (textos formatados em Markdown sem restrição de tamanho, passos práticos e links externos para materiais multimídia). É consumida diretamente pelo backend Java através do endpoint **/aulas/{aulaId}/conteudo**.
3. **Vínculo Lógico Desacoplado:** A rastreabilidade entre as duas coleções e o banco Oracle ocorre através da chave **aula_id** referenciando **AULA.id_aula**. Trata-se de uma associação lógica em nível de aplicação, preservando a independência física dos modelos.

1. Coleção aulas (Metadados Relacionais Exportados)

1.1 Dataset Estruturado em JSON (mongodb/aulas_dataset.json)

Dataset gerado a partir da exportação nativa dos registros reais da tabela **AULA** do Oracle 19c (via **PKG_PETGUARDIAN.pr_exportar_aulas_json**):

```JSON
[
  {
    "id_aula": 1,
    "nome": "Introdução ao Clicker",
    "descricao": "Fundamentos de marcação positiva",
    "pontos_aula": 50,
    "dificuldade": "FACIL",
    "concluida": 1,
    "modulo_id_modulo": 1
  },
  {
    "id_aula": 2,
    "nome": "Critério de Duração do Fica",
    "descricao": "Aumentando o tempo gradativamente",
    "pontos_aula": 80,
    "dificuldade": "MEDIO",
    "concluida": 0,
    "modulo_id_modulo": 1
```

```
  },
  {
    "id_aula": 3,
    "nome": "Troca de Foco na Guia",
    "descricao": "Atenção ao tutor em vez de estímulos da rua",
    "pontos_aula": 70,
    "dificuldade": "MEDIO",
    "concluida": 1,
    "modulo_id_modulo": 2
  },
  {
    "id_aula": 4,
    "nome": "Receitas de Enriquecimento",
    "descricao": "Preparo de petiscos congelados saudáveis",
    "pontos_aula": 40,
    "dificuldade": "FACIL",
    "concluida": 1,
    "modulo_id_modulo": 4
  },
  {
    "id_aula": 5,
    "nome": "Rotas de Fuga e Segurança",
    "descricao": "Evitando que o felino se sinta acuado",
    "pontos_aula": 60,
    "dificuldade": "AVANCADO",
    "concluida": 0,
    "modulo_id_modulo": 5
  },
  {
    "id_aula": 6,
    "nome": "Estímulo Olfativo com Catnip e Silvervine",
    "descricao": "Enriquecimento olfativo para felinos domésticos",
    "pontos_aula": 50,
    "dificuldade": "FACIL",
    "concluida": 1,
    "modulo_id_modulo": 5
  }
]
```


1.2 Script NoSQL de Criação, Validação e Carga (mongodb/setup_mongodb_aulas.js)

JavaScript

```
const log = typeof console !== "undefined" && console.log ? console.log :
print;
```

```
// 1. Seleciona o banco oficial da aplicação
const dbName = "petguardian";
const dbTarget = typeof db !== "undefined" && db.getSiblingDB ?
db.getSiblingDB(dbName) : db;

log(">>> Conectado ao banco: " + dbName);

// 2. Remove coleção preexistente para recriação limpa e idempotente
try {
  dbTarget.aulas.drop();
  log(">>> Coleção antiga 'aulas' removida para recriação limpa.");
} catch (e) {
  // Ignora erro se a coleção ainda não existia
}

// 3. Criação da coleção 'aulas' com JSON Schema Validator
dbTarget.createCollection("aulas", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["id_aula", "nome", "descricao", "pontos_aula", "dificuldade",
"concluida", "modulo_id_modulo"],
      properties: {
        id_aula: {
          bsonType: ["int", "long", "double"],
          description: "Identificador primário da aula de origem no Oracle
(obrigatório e único)."
        },
        nome: {
          bsonType: "string",
          description: "Título da aula pedagógica."
        },
        descricao: {
          bsonType: "string",
          description: "Resumo pedagógico da aula."
        },
        pontos_aula: {
          bsonType: ["int", "long", "double"],
          description: "Pontuação atribuída ao pet após a conclusão da aula."
        },
        dificuldade: {
          bsonType: "string",
          description: "Nível de dificuldade (ex: FACIL, MEDIO, AVANCADO)."
        },
        concluida: {
          bsonType: ["int", "long", "double", "bool"],
          description: "Flag de conclusão da aula (1 concluída, 0 pendente)."
        },
```

```
        modulo_id_modulo: {
          bsonType: ["int", "long", "double"],
          description: "ID do módulo relacional ao qual a aula pertence."
        }
      }
    }
  }
});
log(">>> Coleção 'aulas' criada com JSON Schema Validator.");

// 4. Criação de Índices Estratégicos
dbTarget.aulas.createIndex(
  { "id_aula": 1 },
  { unique: true, name: "idx_id_aula_unique" }
);

dbTarget.aulas.createIndex(
  { "modulo_id_modulo": 1 },
  { name: "idx_modulo_id_modulo" }
);

dbTarget.aulas.createIndex(
  { "dificuldade": 1 },
  { name: "idx_dificuldade" }
);

log(">>> Índices estratégicos criados para a coleção 'aulas'.");

// 5. Carga de Dados a partir de 'aulas_dataset.json' (sem fallbacks embutidos)
const fs = require("fs");
const path = require("path");

const caminhosPossiveis = [
  path.resolve(typeof __dirname !== "undefined" ? __dirname : ".",
"aulas_dataset.json"),
  path.resolve(typeof __dirname !== "undefined" ? __dirname : ".", "mongodb",
"aulas_dataset.json"),
  path.resolve(typeof __dirname !== "undefined" ? __dirname : ".",
"Database-Advanced", "mongodb", "aulas_dataset.json")
];

const caminhoJson = caminhosPossiveis.find(p => fs.existsSync(p));

if (!caminhoJson) {
  throw new Error("ERRO CRÍTICO: Arquivo 'aulas_dataset.json' não encontrado.
Execute a partir da pasta 'mongodb/'.");
}
```

```
const conteudoBruto = fs.readFileSync(caminhoJson, "utf8");
const dadosParsed = JSON.parse(conteudoBruto);

const datasetAulas = dadosParsed.map(item => ({
  ...item,
  id_aula: typeof NumberInt !== "undefined" ? NumberInt(item.id_aula) :
item.id_aula,
  pontos_aula: typeof NumberInt !== "undefined" ? NumberInt(item.pontos_aula) :
item.pontos_aula,
  concluida: typeof NumberInt !== "undefined" ? NumberInt(item.concluida) :
item.concluida,
  modulo_id_modulo: typeof NumberInt !== "undefined" ?
NumberInt(item.modulo_id_modulo) : item.modulo_id_modulo
}));

const resultadoCarga = dbTarget.aulas.insertMany(datasetAulas);
log(">>> Carga executada! Documentos inseridos na coleção 'aulas': " +
(resultadoCarga.insertedCount || Object.keys(resultadoCarga.insertedIds ||
{}).length));

// 6. Consulta de Verificação
log(">>> Consulta de teste (Aulas cadastradas):");
dbTarget.aulas.find({}, {
  id_aula: 1,
  nome: 1,
  pontos_aula: 1,
  dificuldade: 1,
  modulo_id_modulo: 1
}).forEach(doc => {
  log(" - Aula ID: " + doc.id_aula + " | Título: " + doc.nome + " | Pontos: " +
doc.pontos_aula + " | Módulo: " + doc.modulo_id_modulo);
});
```

2. Coleção conteudos_aula (Conteúdo Pedagógico Rico em Markdown)

2.1 Dataset Didático em JSON (mongodb/conteudos_dataset.json)

```
JSON
[
  {
    "aula_id": 1,
    "tipo_conteudo": "PASSO_A_PASSO",
    "corpo_markdown": "# Introdução ao Clicker\n\nFundamentos de marcação
positiva utilizando o método de indução positiva com petisco.\n\n### Passo a
```

```
Passo:\n1. Carregue o clicker com petisco de alto valor.\n2. Clique no exato instante em que o cão oferecer a resposta desejada.\n3. Entregue o petisco em até 2 segundos para fixação cognitiva.\n\n> **Dica de Ouro:** Repita em sessões de no máximo 3 minutos para não causar estresse cognitivo.",
    "links_recursos": [

"https://www.akc.org/expert-advice/training/clicker-training-your-dog-mark-and-reward/",
      "https://en.wikipedia.org/wiki/Clicker_training"
    ],
    "atualizado_em": "2026-10-08T18:00:00Z"
  },
  {
    "aula_id": 2,
    "tipo_conteudo": "ARTIGO",
    "corpo_markdown": "# Critério de Duração do Fica\n\nAumentando o tempo gradativamente sem que o cão quebre a posição de espera.\n\n### Princípios Fundamentais:\n- Inicie com 3 segundos de permanência.\n- Dê um passo para trás e retorne antes de liberar o animal com comando de liberação ('Ok!').\n- Jamais chame o cão até você quando ele estiver no comando Fica para não gerar antecipação de quebra de postura.",
    "links_recursos": [
      "https://www.rspca.org.uk/adviceandwelfare/pets/dogs/training/stay"
    ],
    "atualizado_em": "2026-10-08T18:00:00Z"
  },
  {
    "aula_id": 3,
    "tipo_conteudo": "VIDEO",
    "corpo_markdown": "# Troca de Foco na Guia\n\nAtenção ao tutor em vez de estímulos da rua durante os passeios urbanos.\n\n### Procedimento Prático:\n1. Use guia de 1,5m sem tensionar.\n2. Sempre que avistar estímulo concorrente na calçada, chame o cão e recompense o contato visual imediato.\n3. Mantenha caminhada fluida e recompense o caminhar ao lado.",
    "links_recursos": [
      "https://www.akc.org/expert-advice/training/teach-puppy-walk-leash/"
    ],
    "atualizado_em": "2026-10-08T18:00:00Z"
  },
  {
    "aula_id": 4,
    "tipo_conteudo": "PASSO_A_PASSO",
    "corpo_markdown": "# Receitas de Enriquecimento\n\nPreparo de petiscos congelados saudáveis para enriquecimento ambiental e redução de ansiedade de separação.\n\n### Receita Kong Calmante:\n- 1 colher de pasta de amendoim 100% integral (sem xilitol e sem açúcar).\n- Frutas picadas permitidas (maçã sem semente e banana amassada).\n- Congele por 4 horas no freezer antes de oferecer ao animal.",
```

```
    "links_recursos": [
      "https://www.kongcompany.com/recipes/"
    ],
    "atualizado_em": "2026-10-08T18:00:00Z"
  },
  {
    "aula_id": 5,
    "tipo_conteudo": "ARTIGO",
    "corpo_markdown": "# Rotas de Fuga e Segurança\n\nEvitando que o felino se
sinta acuado em ambientes verticais para felinos.\n\n### Recomendações de
Instalação:\n- Instale prateleiras e nichos a pelo menos 1,5m de altura com
superfície antiderrapante.\n- Nunca crie rotas sem saída ('becos sem saída
verticais') em lares com mais de um gato.\n- Garanta redes de proteção testadas
em 100% das janelas e sacadas.",
    "links_recursos": [

"https://www.vet.cornell.edu/departments-centers-and-institutes/cornell-feline-
health-center/health-information/feline-health-topics/feline-behavior-problems-
destructive-behavior"
    ],
    "atualizado_em": "2026-10-08T18:00:00Z"
  },
  {
    "aula_id": 6,
    "tipo_conteudo": "ARTIGO",
    "corpo_markdown": "# Estímulo Olfativo com Catnip e
Silvervine\n\nEnriquecimento olfativo para felinos domésticos para estímulo
mental e prevenção de apatia.\n\n### Guia de Aplicação:\n- **Catnip (Nepeta
cataria):** aplicar 1 a 2 vezes por semana em brinquedos ou arranhadores.\n-
**Silvervine (Actinidia polygama):** opção consagrada para os 30% de felinos
que não respondem ao Catnip.\n- Guarde os brinquedos em potes herméticos após a
brincadeira para preservar os óleos essenciais.",
    "links_recursos": [
      "https://en.wikipedia.org/wiki/Catnip",
      "https://en.wikipedia.org/wiki/Actinidia_polygama"
    ],
    "atualizado_em": "2026-10-08T18:00:00Z"
  }
]
```


2.2 Script NoSQL de Criação, Validação e Carga (mongodb/setup_mongodb_conteudos.js)

JavaScript

```
// ======================================================================
// CHALLENGE CLYVO 2026 – SPRINT 4
// DISCIPLINA: Mastering Relational and Non-Relational Database
// REQUISITO 4: Estrutura MongoDB (Código de Criação, Validação e Índices)
// TURMA: 2TDSPG
//
// COMO EXECUTAR:
// 1. Via Linha de Comando (mongosh) a partir da pasta 'mongodb/':
//    cd mongodb
//    mongosh "mongodb+srv://<usuario>:<senha>@cluster.mongodb.net/petguardian"
setup_mongodb_conteudos.js
// 2. No MongoDB Compass (aba ">_MONGOSH"):
//    process.chdir("<caminho_do_repositorio>/Database-Advanced/mongodb")
//    load("setup_mongodb_conteudos.js")
// ======================================================================

// Função de log universal (compatível com mongosh, mongo shell e Node.js)
const log = typeof console !== "undefined" && console.log ? console.log :
print;

// 1. Seleciona ou cria o banco de dados oficial da aplicação
const dbName = "petguardian";
const dbTarget = typeof db !== "undefined" && db.getSiblingDB ?
db.getSiblingDB(dbName) : db;

log(">>> Conectado ao banco: " + dbName);

// 2. Remove coleção preexistente se necessário para execução limpa e repetível
try {
  dbTarget.conteudos_aula.drop();
  log(">>> Coleção antiga 'conteudos_aula' removida para recriação limpa.");
} catch (e) {
  // Ignora erro se a coleção ainda não existia
}

// 3. Criação da Coleção com Validação de Schema (JSON Schema Validator)
// Garante integridade estrutural e tipagem dos documentos NoSQL
dbTarget.createCollection("conteudos_aula", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["aula_id", "tipo_conteudo", "corpo_markdown"],
      properties: {
        aula_id: {
          bsonType: ["int", "long", "string"],
          description: "ID relacional da aula de origem no Oracle (inteiro/long
ou GUID string obrigatório e único)."
```

```
        },
        tipo_conteudo: {
          bsonType: "string",
          description: "Classificação do formato pedagógico (ex: ARTIGO, VIDEO,
PASSO_A_PASSO)."
        },
        corpo_markdown: {
          bsonType: "string",
          description: "Conteúdo rico formatado em Markdown, sem restrição de
tamanho."
        },
        links_recursos: {
          bsonType: "array",
          description: "Lista de URLs para materiais complementares e vídeos de
apoio.",
          items: {
            bsonType: "string"
          }
        },
        atualizado_em: {
          bsonType: ["string", "date"],
          description: "Data/hora ISO da última revisão do conteúdo."
        }
      }
    }
  }
});
log(">>> Coleção 'conteudos_aula' criada com JSON Schema Validator.");

// 4. Criação de Índices Estratégicos para Otimização NoSQL
// Índice 1: Unicidade no ID da aula relacional (idempotência e integridade
lógica)
dbTarget.conteudos_aula.createIndex(
  { "aula_id": 1 },
  { unique: true, name: "idx_aula_id_unique" }
);

// Índice 2: Índice Textual para buscas por palavras-chave dentro do conteúdo
pedagógico
dbTarget.conteudos_aula.createIndex(
  { "corpo_markdown": "text" },
  { name: "idx_busca_textual_conteudo" }
);

// Índice 3: Índice de filtro por formato de conteúdo
dbTarget.conteudos_aula.createIndex(
  { "tipo_conteudo": 1 },
  { name: "idx_tipo_conteudo" }
```

```
);

log(">>> Índices estratégicos criados com sucesso.");

// 5. Carga dos Dados a partir do Dataset Estruturado JSON
(conteudos_dataset.json)
const fs = require("fs");
const path = require("path");

const caminhosPossiveis = [
  path.resolve(typeof __dirname !== "undefined" ? __dirname : ".",
"conteudos_dataset.json"),
  path.resolve(typeof __dirname !== "undefined" ? __dirname : ".", "mongodb",
"conteudos_dataset.json"),
  path.resolve(typeof __dirname !== "undefined" ? __dirname : ".",
"Database-Advanced", "mongodb", "conteudos_dataset.json")
];

const caminhoJson = caminhosPossiveis.find(p => fs.existsSync(p));

if (!caminhoJson) {
  throw new Error("ERRO CRÍTICO: Arquivo 'conteudos_dataset.json' não
encontrado. Execute a partir da pasta 'mongodb/'.");
}

const conteudoBruto = fs.readFileSync(caminhoJson, "utf8");
const dadosParsed = JSON.parse(conteudoBruto);

const datasetConteudos = dadosParsed.map(item => ({
  ...item,
  aula_id: typeof NumberInt !== "undefined" ? NumberInt(item.aula_id) :
item.aula_id,
  atualizado_em: item.atualizado_em || new Date().toISOString()
}));
log(">>> Sucesso: Dataset carregado dinamicamente a partir de
'conteudos_dataset.json' (" + datasetConteudos.length + " documentos).");

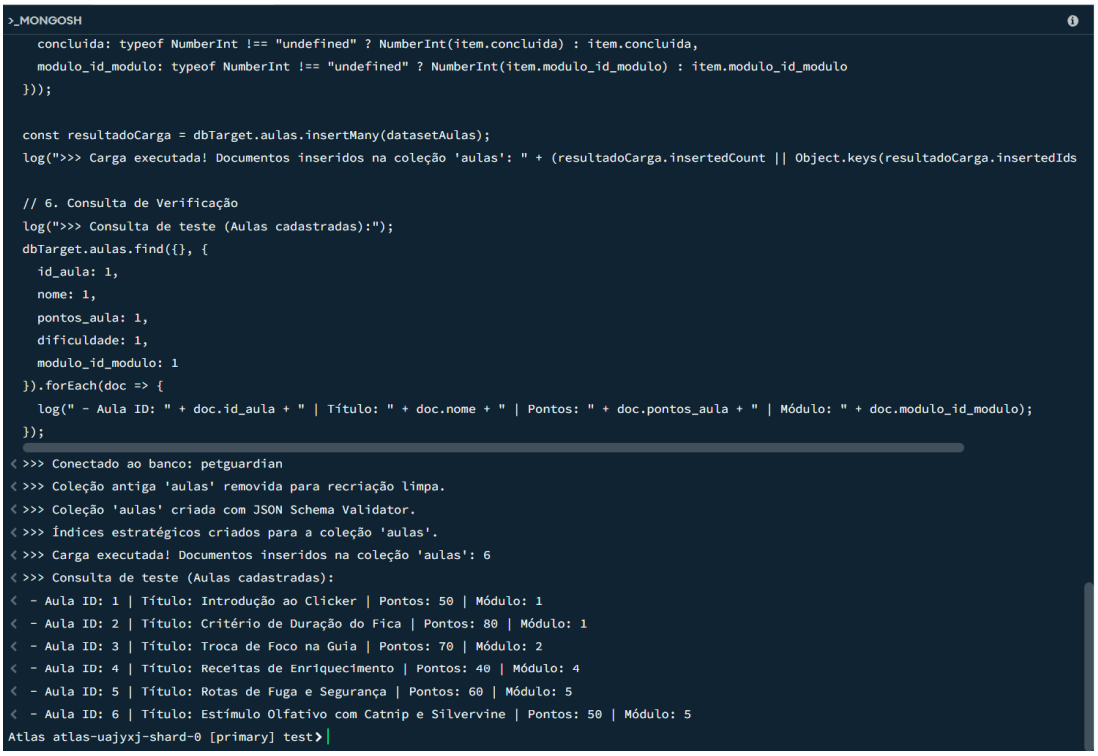
const resultadoCarga = dbTarget.conteudos_aula.insertMany(datasetConteudos);
log(">>> Carga executada! Documentos inseridos com sucesso: " +
(resultadoCarga.insertedCount || Object.keys(resultadoCarga.insertedIds ||
{}).length));

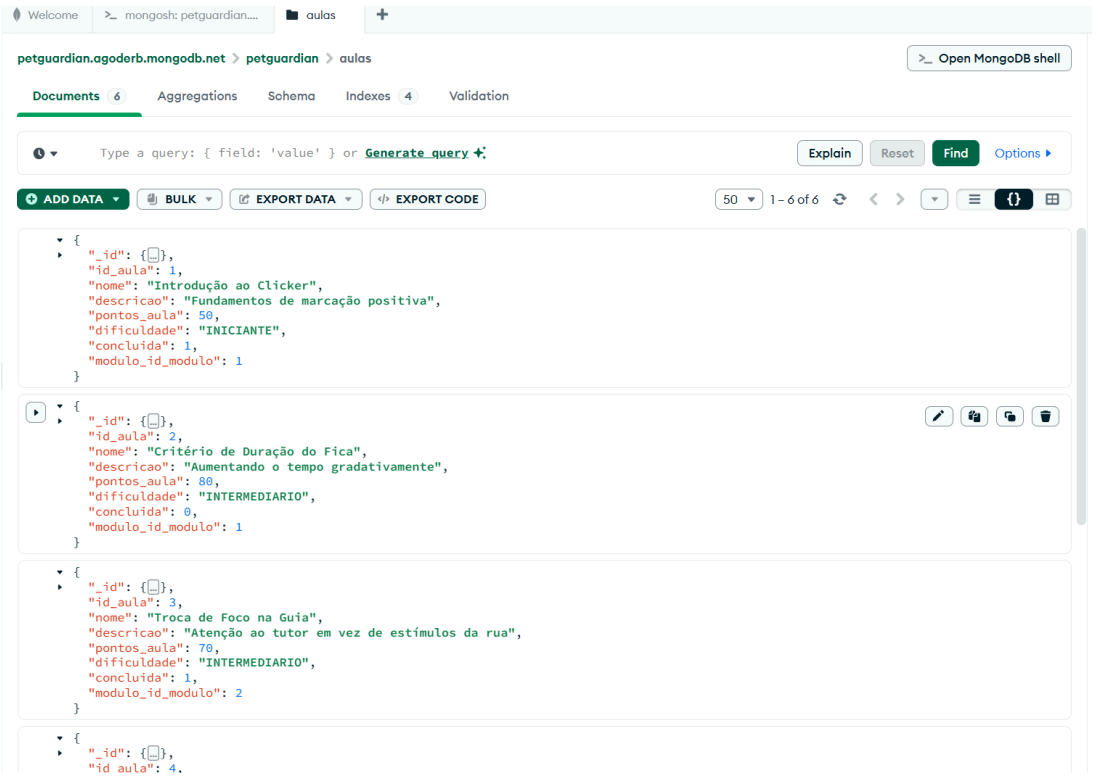
// 6. Consulta de Verificação com Projeção
log(">>> Consulta de teste (Conteúdos cadastrados):");
dbTarget.conteudos_aula.find({}, {
  aula_id: 1,
  tipo_conteudo: 1,
  atualizado_em: 1
```

```
}).forEach(doc => {
  log(" - Aula ID: " + doc.aula_id + " | Formato: " + doc.tipo_conteudo);
});
```

3. Evidências de Execução e Importação no MongoDB (MONGOSH / Compass)

Print da execução de setup_mongodb_aulas.js no terminal MONGOSH exibindo o log de criação da coleção 'aulas', índices e documentos inseridos





Print da execução de setup_mongodb_conteudos.js no terminal MONGOSH exibindo o log de criação da coleção 'conteudos_aula', índices e documentos inseridos

```
>_MONGOSH
    aula_id: typeof NumberInt !== "undefined" ? NumberInt(item.aula_id) : item.aula_id,
    atualizado_em: item.atualizado_em || new Date().toISOString()
  }));
  log(">>> Sucesso: Dataset carregado dinamicamente a partir de 'conteudos_dataset.json' (" + datasetConteudos.length + " documentos).");

  const resultadoCarga = dbTarget.conteudos_aula.insertMany(datasetConteudos);
  log(">>> Carga executada! Documentos inseridos com sucesso: " + (resultadoCarga.insertedCount || Object.keys(resultadoCarga.insertedIds || {})

  // 6. Consulta de Verificação com Projeção
  log(">>> Consulta de teste (Conteúdos cadastrados):");
  dbTarget.conteudos_aula.find({}, {
    aula_id: 1,
    tipo_conteudo: 1,
    atualizado_em: 1
  }).forEach(doc => {
    log(" - Aula ID: " + doc.aula_id + " | Formato: " + doc.tipo_conteudo);
  });
< >>> Conectado ao banco: petguardian
< >>> Coleção antiga 'conteudos_aula' removida para recriação limpa.
< >>> Coleção 'conteudos_aula' criada com JSON Schema Validator.
< >>> Índices estratégicos criados com sucesso.
< >>> Sucesso: Dataset carregado dinamicamente a partir de 'conteudos_dataset.json' (6 documentos).
< >>> Carga executada! Documentos inseridos com sucesso: 6
< >>> Consulta de teste (Conteúdos cadastrados):
<  - Aula ID: 1 | Formato: PASSO_A_PASSO
<  - Aula ID: 2 | Formato: ARTIGO
<  - Aula ID: 3 | Formato: VIDEO
<  - Aula ID: 4 | Formato: PASSO_A_PASSO
<  - Aula ID: 5 | Formato: ARTIGO
<  - Aula ID: 6 | Formato: ARTIGO
Atlas atlas-uajyxj-shard-0 [primary] test>
```

Welcome | mongosh: petguardian.... | conteudos_aula

petguardian.agoderb.mongodb.net > petguardian > conteudos_aula

Open MongoDB shell

Documents 6 | Aggregations | Schema | Indexes 4 | Validation

Type a query: { field: 'value' } or Generate query

Explain | Reset | Find | Options

ADD DATA | BULK | EXPORT DATA | EXPORT CODE

50 | 1 - 6 of 6


```
{
  "_id": {...},
  "aula_id": 1,
  "tipo_conteudo": "PASSO_A_PASSO",
  "corpo_markdown": "# Introdução ao Clicker\n\nFundamentos de marcação positiva utilizando o método de indução positiva com petisco.\n
  "links_recursos": [...],
  "atualizado_em": "2026-10-08T18:00:00Z"
}
{
  "_id": {...},
  "aula_id": 2,
  "tipo_conteudo": "ARTIGO",
  "corpo_markdown": "# Critério de Duração do Fica\n\nAumentando o tempo gradativamente sem que o cão quebre a posição de espera.\n\n##
  "links_recursos": [...],
  "atualizado_em": "2026-10-08T18:00:00Z"
}
{
  "_id": {...},
  "aula_id": 3,
  "tipo_conteudo": "VIDEO",
  "corpo_markdown": "# Troca de Foco na Guia\n\nAtenção ao tutor em vez de estímulos da rua durante os passeios urbanos.\n\n### Procedi
  "links_recursos": [...],
  "atualizado_em": "2026-10-08T18:00:00Z"
}
{
  "_id": {...},
  "aula_id": 4,
  "tipo_conteudo": "PASSO_A_PASSO",
  "corpo_markdown": "# Receitas de Enriquecimento\n\nPreparo de petiscos congelados saudáveis para enriquecimento ambiental e redução d
  "links_recursos": [...],
  "atualizado_em": "2026-10-08T18:00:00Z"
```